板子

目录

- 背包DP
- 区间DP
- 数位DP

数学

- 数学基础
- 组合数
- Lucas定理
- 线性筛
- 矩阵快速幂
- 矩阵类
- EXGCD
- 中国剩余定理
- 高斯消元
- 斯特林数
- 整除分块
- 线性基
- 模整数类
- FFT
- NTT
- FWT
- 多项式
- 多项式插值
- 离散对数
- GF64
- Lehmer 码
- 类欧几里得算法

计算几何

- 基本数据类型
- 平面凸包
- 最近点对
- 半平面交
- 扫描线
- Pick定理
- 圆与多边形相交
- 圆与圆相交
- 凸多边形
- KDTree
- 李超线段树

字符串

- KMP
- Manacher
- 字典树
- 字符串哈希
- AC自动机
- 后缀数组
- Height数组
- 后缀自动机
- Lyndon分解
- 最小表示法
- 序列自动机
- 回文自动机
- Z函数

其他

- 最长上升子序列
- 高精度整数
- 快速读入
- SplitMix64 自定义哈希

让代码跑起来

编译运行

```
g++ -std=c++20 -O2 -DLOCAL $1.cpp -o $1
ulimit -s unlimited
if [[ -z $2 ]]; then
    ./$1
else
    ./$1 < $2
fi
```

```
param($name, $inp = '')
g++ "$name.cpp" -o "$name.exe" -std=c++20 -O2 "-Wl,--stack,100000000" -DLOCAL
if(!$inp) {
    & "./$name.exe"
} else {
    gc "$inp" | & "./$name.exe"
}
```

GDB

```
g++ -std=c++20 -O0 -g $1.cpp -o $1
gdb $1


# 打开pretty print
set print pretty on
set pagination off
set print element 0
# 运行
run
# 如果从文件读取输入
# run < input.txt
# 断点
b main # main函数
b 23 # 23行
i b # 查看断点
delete 1 # 删除1号断点
disable 1 # 禁用1号断点
enable 1 # 启用1号断点
b dfs if u == 21 && fa != -1 # 条件断点
condition 1 argc != 1 # 更改为条件断点
condition 2 # 更改为无条件
c # 在断点处暂停后，继续运行，直到走到下一个断点
n # 单步执行，不进入函数
s # 单步执行，进入函数
# 打印数据
p x # 输出x的值
p arr[0]@10 # 打印arr[0]到arr[0+10-1]
watch x # 当暂停时显示
display x # 当x更改时显示
# 调用函数
p nums.size()
call nums.push_back(2)
# 设置变量
set var x = 5 # 给x赋值为5
# 查看调用栈
bt
bt full # 打印调用栈的同时查看局部变量
f 2 # 进入第 2 层（切换栈帧）
info args # 查看参数
info locals # 查看局部变量
```

对拍器

Linux Shell

```
g++ gen.cpp -o gen -std=c++20 -O2 || exit 1
g++ sol1.cpp -o sol1 -std=c++20 -O2 || exit 1
g++ sol2.cpp -o sol2 -std=c++20 -O2 || exit 1
mkdir -p tc
ulimit -s unlimited
cnt=1
while [ $cnt -le 100000 ]; do
    echo "Running $cnt"
    ./gen > tc/input.txt
    timeout 2s ./sol1 < tc/input.txt > tc/sol1.txt || exit 2
    timeout 2s ./sol2 < tc/input.txt > tc/sol2.txt || exit 2
    if ! diff -wB tc/sol1.txt tc/sol2.txt > /dev/null; then
        echo -e "Wrong Answer on $cnt"
        echo "Input:"
        cat tc/input.txt
        echo -e "\nSol1:"
        cat tc/sol1.txt
        echo -e "\nSol2:"
        cat tc/sol2.txt
        exit 1
    fi
    echo -e "Accepted"
    ((cnt++))
done
```

Windows PowerShell

```
sal p Write-Host
g++ gen.cpp -o gen -std=c++20 -O2 "-Wl,--stack,100000000"
g++ sol.cpp -o sol -std=c++20 -O2 "-Wl,--stack,100000000"
g++ sol2.cpp -o sol2 -std=c++20 -O2 "-Wl,--stack,100000000"
mkdir -p tc -f
$cnt = 1
while($cnt -le 100000) {
    p Running $cnt
    ./gen > tc/input.txt
    gc tc/input.txt -ra | ./sol > tc/sol.txt
    gc tc/input.txt -ra | ./sol2 > tc/sol2.txt
    $f1 = (gc tc/sol.txt -ra) -replace  '\s+', ''
    $f2 = (gc tc/sol2.txt -ra) -replace '\s+', ''
    if($f1 -ne $f2) {
        p "Wrong answer" -f r
        p Input:
        gc tc/input.txt -ra
        p Sol1:
        gc tc/sol.txt -ra
        p Sol2:
        gc tc/sol2.txt -ra
        break
    }
    p Accepted -f gre
    $cnt++
}
```

交互器

```
from argparse import *
from subprocess import *
import sys

ap = ArgumentParser()
ap.add_argument('program', nargs='+')
ap.add_argument('-x', '--num-to-guess', type=int, dest='x', default=42)
ap.add_argument('-q', '--quiet', dest='quiet', action='store_true')
args = ap.parse_args()

def write(p: Popen[str], line):
    assert p.poll() is None and p.stdin is not None
    if not args.quiet:
        print(f'Write: {line}', flush=True)
    p.stdin.write(f"{line}\n")
    p.stdin.flush()
def read(p: Popen[str]) -> str:
    assert p.poll() is None and p.stdout is not None
    line = p.stdout.readline().strip()
    assert line != ''
    if not args.quiet:
        print(f'Read: {line}', flush=True)
    return line
def wrong(p: Popen[str], msg):
    p.kill()
    print(f"Wrong answer: {msg}")
    sys.exit(1)

q = 0
with Popen(
    " ".join(args.program), shell=True,
    stdin=PIPE, stdout=PIPE, universal_newlines=True
) as p:
    while True:
        q += 1
        if q > 50:
            wrong(p, "too many queries")
        try:
            s = read(p)
            guess = int(s)
        except:
            wrong(p, f"invalid input: {s}")
        if guess == args.x:
```

```
        write(p, '=')
        break
    elif guess > args.x:
        write(p, '>')
    else:
        write(p, '<')
sys.stdout.write("Accepted")
sys.stdout.write(f"Number of query: {q}\n")
sys.stdout.flush()
```

数据结构

并查集

```
struct DSU {
    explicit DSU(int n) : fs(n, -1) {}
    int find(int x) {
        if(fs[x] < 0) return x;
        return fs[x] = find(fs[x]);
    }
    bool is_connected(int x, int y) {
        return find(x) == find(y);
    }
    int size(int x) {
        return -fs[find(x)];
    }
    void connect(int x, int y) {
        x = find(x);
        y = find(y);
        if(x == y) return;
        int sx = size(x), sy = size(y);
        if(sx < sy) {
            fs[y] -= sx;
            fs[x] = y;
        } else {
            fs[x] -= sy;
            fs[y] = x;
        }
    }
    // make fa[find(y)] = find(x)
    void directed_connect(int x, int y) {
        x = find(x);
        y = find(y);
        if(x == y) return;
        fs[x] -= fs[y];
        fs[y] = x;
    }

private:
    vector<int> fs; // fa or size
};
```

树的倍增数组

```
struct BinaryLift {
    explicit BinaryLift(int n) : tree(n), anc(n), dep(n) {
        for(auto &a : anc) a.fill(-1);
    }
    void add_edge(int u, int v) {
        tree[u].push_back(v);
        tree[v].push_back(u);
    }
    void build(int u = 0) {
        [&](auto &&dfs) {
            dfs(dfs, u, -1);
        }([&](auto &&dfs, int u, int fa) -> void {
            dep[u] = fa != -1 ? dep[fa] + 1 : 0;
            anc[u][0] = fa;
            for(int k = 1; k < LOG; ++k) {
                if(anc[u][k - 1] == -1) {
                    anc[u][k] = -1;
                } else {
                    anc[u][k] = anc[anc[u][k - 1]][k - 1];
                }
            }
            for(int v : tree[u]) {
                if(v != fa) {
                    dfs(dfs, v, u);
                }
            }
        });
    }
    int lca(int u, int v) const {
        if(dep[u] < dep[v]) swap(u, v);
        for(int k = LOG - 1; k >= 0; --k) {
            if(anc[u][k] != -1) {
                if(dep[anc[u][k]] >= dep[v]) {
                    u = anc[u][k];
                }
            }
        }
        if(u == v) return u;
        for(int k = LOG - 1; k >= 0; --k) {
            if(anc[u][k] != anc[v][k]) {
                u = anc[u][k];
                v = anc[v][k];
            }
```

```
        }
        return anc[u][0];
    }
    int kth_ancestor(int x, int k) const {
        for(int i = 0; i < LOG; ++i) {
            if((k >> i) & 1) {
                x = anc[x][i];
                if(x == -1) return -1;
            }
        }
        return x;
    }
    vector<vector<int>> tree;

private:
    static constexpr int LOG = 20;
    vector<array<int, LOG>> anc;
    vector<int> dep;
};
```

树链剖分

```
struct HLD {
    explicit HLD(int n)
        : tree(n), dfn(n), dep(n), fa(n), toc(n), sz(n), hs(n, -1) {}
    explicit HLD(const vector<vector<int>> &edges, int r = 0)
        : tree(edges), dfn(edges.size()), dep(edges.size()), fa(edges.size()),
          toc(edges.size()), sz(edges.size()), hs(edges.size(), -1) {
        build(r);
    }
    void add_edge(int u, int v) {
        tree[u].push_back(v);
        tree[v].push_back(u);
    }
    void build(int r = 0) {
        [&](auto &&dfs) {
            dfs(dfs, r, -1);
        }([&](auto &&dfs, int x, int f) -> void {
            dep[x] = f == -1 ? 0 : dep[f] + 1;
            fa[x] = f;
            sz[x] = 1;
            hs[x] = -1;
            for(int next : tree[x]) {
                if(next == f) continue;
                dfs(dfs, next, x);
                sz[x] += sz[next];
                if(hs[x] == -1 || sz[hs[x]] < sz[next]) {
                    hs[x] = next;
                }
            }
        });
        int now = 0;
        [&](auto &&dfs) {
            dfs(dfs, r, -1, r);
        }([&](auto &&dfs, int x, int f, int top) -> void {
            dfn[x] = now++;
            toc[x] = top;
            if(hs[x] == -1) return;
            dfs(dfs, hs[x], x, top);
            for(int next : tree[x]) {
                if(next != hs[x] && next != f) {
                    dfs(dfs, next, x, next);
                }
            }
        });
```

```
    }
    int lca(int u, int v) const {
        while(toc[u] != toc[v]) {
            if(dep[toc[u]] < dep[toc[v]]) {
                v = fa[toc[v]];
            } else {
                u = fa[toc[u]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    }
    // 返回在这个路径上的连续区间，注意左闭右开
    vector<pair<int, int>> query_path(int u, int v) const {
        vector<pair<int, int>> up, down;
        while(toc[u] != toc[v]) {
            if(dep[toc[u]] < dep[toc[v]]) {
                down.emplace_back(dfn[toc[v]], dfn[v] + 1);
                v = fa[toc[v]];
            } else {
                up.emplace_back(dfn[toc[u]], dfn[u] + 1);
                u = fa[toc[u]];
            }
        }
        if(dep[u] < dep[v]) {
            down.emplace_back(dfn[u], dfn[v] + 1);
        } else {
            up.emplace_back(dfn[v], dfn[u] + 1);
        }
        reverse(down.begin(), down.end());
        up.insert(up.end(), down.begin(), down.end());
        return up;
    }
    // 返回子树的连续区间，注意左闭右开
    pair<int, int> query_subtree(int x) const {
        return {dfn[x], dfn[x] + sz[x]};
    }
    int size(int x) const {
        return sz[x];
    }

    vector<vector<int>> tree;
    vector<int> dfn, dep, fa, toc, sz, hs;
};
```

树状数组

```cpp
template<class T>
concept FenwickInfo = requires(T a, T b) {
    T{};
    { a + b } -> convertible_to<T>;
};
template<FenwickInfo T>
struct Fenwick {
    explicit Fenwick(int _n) : nums(_n + 1, 0), n(_n) {}
    T query(int x) const {
        T ans{};
        for(; x; x -= x & -x) ans = ans + nums[x];
        return ans;
    }
    void update(int x, const T &v) {
        for(; x <= n; x += x & -x) nums[x] = nums[x] + v;
    }

private:
    vector<T> nums;
    int n;
};
template<class T>
concept RangeFenwickInfo = requires(T a, T b, int c) {
    { a - b } -> convertible_to<T>;
    { a * c } -> convertible_to<T>;
    { -a } -> convertible_to<T>;
} && FenwickInfo<T>;
template<RangeFenwickInfo T>
struct RangeFenwick {
    explicit RangeFenwick(int _n) : f1(_n), f2(_n), n(_n) {}
    void update(int l, int r, const T &v) {
        _update(l, v);
        _update(r, -v);
    }
    T query(int x) const {
        return x * f1.query(x) - f2.query(x);
    }

private:
    Fenwick<T> f1, f2;
    int n;
    void _update(int x, const T &v) {
        f1.update(x, v);
```

```
        f2.update(x, v * (x - 1));
    }
};
```

线段树

```
template<class Info, class Applier>
concept SegInfo =
    requires(Info a, Info b, const Applier src, vector<Info> vi, int u) {
        Info{};
        Info(a);
        { a + b } -> same_as<Info>;
        { src.apply(a) } -> same_as<void>;
        { vi[u] } -> same_as<Info &>;
    };

template<class Info, class Applier>
    requires(SegInfo<Info, Applier>)
struct SegTree {
    SegTree() : n(0) {}
    explicit SegTree(int sz) : n(sz), info(sz * 4, Info()) {}
    explicit SegTree(const vector<Info> &v) : n(v.size()), info(v.size() * 4) {
        [&](auto &&bld) {
            bld(bld, v, 0, 0, n);
        }([&](auto &&bld, const vector<Info> &v, int u, int rl,
              int rr) -> void {
            if(rl == rr - 1) {
                info[u] = v[rl];
                return;
            }
            int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
            bld(bld, v, ls, rl, mid);
            bld(bld, v, rs, mid, rr);
            info[u] = info[ls] + info[rs];
        });
    }
    Info query(int l, int r) const {
        if(l >= r) return Info();
        return [&](auto &&qry) {
            return qry(qry, l, r, 0, 0, n);
        }([&](auto &&qry, int ql, int qr, int u, int rl, int rr) -> Info {
            if(ql <= rl && qr >= rr) return info[u];
            int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
            Info ret{};
            if(ql < mid) ret = ret + qry(qry, ql, qr, ls, rl, mid);
            if(qr > mid) ret = ret + qry(qry, ql, qr, rs, mid, rr);
            return ret;
        });
    }
```

```
    void update(int x, const Applier &v) {
        [&](auto &&upd) {
            upd(upd, x, v, 0, 0, n);
        }([&](auto &&upd, int x, const Applier &v, int u, int rl,
              int rr) -> void {
            if(rl == rr - 1) {
                v.apply(info[u]);
                return;
            }
            int mid = (rl + rr) >> 1, ls = u * 2 + 1, rs = u * 2 + 2;
            if(x < mid) upd(upd, x, v, ls, rl, mid);
            else upd(upd, x, v, rs, mid, rr);
            info[u] = info[ls] + info[rs];
        });
    }

private:
    int n;
    vector<Info> info;
};
```

懒标记线段树

```
template<class Info, class Tag>
concept SegInfoTag = requires(Info a, Info b, Tag c, Tag d, int l, int r,
                              vector<Info> vi, vector<Tag> vt) {
    Info{};
    Tag{};
    { a + b } -> same_as<Info>;
    { c.apply(a, l, r) } -> same_as<void>;
    { c.apply(d, l, r) } -> same_as<void>;
    { c.empty() } -> same_as<bool>;
    { c.clear() } -> same_as<void>;
    { vi[l] } -> same_as<Info &>;
    { vt[l] } -> same_as<Tag &>;
};
template<class Info, class Tag>
    requires(SegInfoTag<Info, Tag>)
struct LazySegTree {
    LazySegTree() : n(0) {}
    explicit LazySegTree(int n_) : n(n_), info(4 * n_), tag(4 * n_) {}
    explicit LazySegTree(const vector<Info> &v)
        : n(v.size()), info(4 * v.size()), tag(4 * v.size()) {
        [&](auto &&bld) {
            bld(bld, v, 0, 0, n);
        }([&](auto &&bld, const vector<Info> &v, int u, int rl,
              int rr) -> void {
            if(rr - rl == 1) {
                info[u] = v[rl];
                return;
            }
            int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
            bld(bld, v, ls, rl, mid);
            bld(bld, v, rs, mid, rr);
            info[u] = info[ls] + info[rs];
        });
    }
    Info query(int l, int r) {
        return [&](auto &&qry) {
            return qry(qry, l, r, 0, 0, n);
        }([&](auto &&qry, int ql, int qr, int u, int rl, int rr) -> Info {
            if(ql <= rl && qr >= rr) return info[u];
            _pushdown(u, rl, rr);
            Info ret{};
            int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
            if(ql < mid) ret = ret + qry(qry, ql, qr, ls, rl, mid);
```

```
            if(qr > mid) ret = ret + qry(qry, ql, qr, rs, mid, rr);
            return ret;
        });
    }
    void update(int l, int r, const Tag &t) {
        [&](auto &&upd) {
            upd(upd, l, r, t, 0, 0, n);
        }([&](auto &&upd, int ul, int ur, const Tag &t, int u, int rl,
              int rr) -> void {
            if(ul <= rl && ur >= rr) {
                t.apply(info[u], rl, rr);
                t.apply(tag[u], rl, rr);
                return;
            }
            _pushdown(u, rl, rr);
            int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
            if(ul < mid) upd(upd, ul, ur, t, ls, rl, mid);
            if(ur > mid) upd(upd, ul, ur, t, rs, mid, rr);
            info[u] = info[ls] + info[rs];
        });
    }

private:
    int n;
    vector<Info> info;
    vector<Tag> tag;
    void _pushdown(int u, int rl, int rr) {
        if(tag[u].empty()) return;
        int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        tag[u].apply(info[ls], rl, mid);
        tag[u].apply(info[rs], mid, rr);
        tag[u].apply(tag[ls], rl, mid);
        tag[u].apply(tag[rs], mid, rr);
        tag[u].clear();
    }
};
```

值域线段树

```
struct ValSeg {
    ValSeg(int ma) : s((ma + 1) * 4), n(ma + 1) {}
    ValSeg() : n(0) {}
    void assign(int ma) {
        s.assign((ma + 1) * 4, 0);
        n = ma + 1;
    }
    void insert(int x) {
        update(x, 1);
    }
    void erase(int x) {
        update(x, -1);
    }
    int qpre(int x) const {
        int rk = qrv(x);
        return qvr(rk - 1);
    }
    int qsuc(int x) const {
        int rk = qrv(x + 1);
        return qvr(rk);
    }
    int qrv(int x) const {
        return qrange_cnt(0, x);
    }
    int qvr(int k) const {
        return [&](auto &&qry) {
            return qry(qry, k, 0, 0, n);
        }([&](auto &&qry, int k, int rt, int rl, int rr) -> int {
            if(rr - rl == 1) return rl;
            int m = (rl + rr) >> 1, ls = rt * 2 + 1, rs = rt * 2 + 2;
            if(k < s[ls]) return qry(qry, k, ls, rl, m);
            return qry(qry, k - s[ls], rs, m, rr);
        });
    }
    int size() const {
        return s[0];
    }
    int qcnt(int x) const {
        return qrange_cnt(x, x + 1);
    }
    int qrange_cnt(int l, int r) const {
        return [&](auto &&qry) {
            return qry(qry, l, r, 0, 0, n);
```

```
        }([&](auto &&qry, int ql, int qr, int rt, int rl, int rr) -> int {
            if(ql <= rl && qr >= rr) return s[rt];
            int m = (rl + rr) >> 1, ls = rt * 2 + 1, rs = rt * 2 + 2;
            int ans = 0;
            if(ql < m) ans += qry(qry, ql, qr, ls, rl, m);
            if(qr > m) ans += qry(qry, ql, qr, rs, m, rr);
            return ans;
        });
    }

private:
    vector<int> s;
    int n;
    void update(int x, int dv) {
        [&](auto &&upd) {
            upd(upd, x, dv, 0, 0, n);
        }([&](auto &&upd, int x, int dv, int rt, int rl, int rr) -> void {
            if(rr - rl == 1) {
                s[rt] += dv;
                return;
            }
            int m = (rl + rr) >> 1, ls = rt * 2 + 1, rs = rt * 2 + 2;
            if(x < m) upd(upd, x, dv, ls, rl, m);
            else upd(upd, x, dv, rs, m, rr);
            s[rt] = s[ls] + s[rs];
        });
    }
};
```

动态开点线段树

```
template<class Info, class Applier>
concept DynSegInfo = requires(Info a, Info b, const Applier src) {
    Info{};
    Info(a);
    { a + b } -> same_as<Info>;
    { src.apply(a) } -> same_as<void>;
};

template<class Info, class Applier>
    requires(DynSegInfo<Info, Applier>)
struct DynSegTree {
    explicit DynSegTree(ll _n) : n(_n), info(1, _(0, _n)) {}
    void update(ll x, const Applier &src) {
        [&](auto &&upd) {
            upd(upd, x, src, 0);
        }([&](auto &&upd, ll x, const Applier &src, ll u) -> void {
            if(info[u].l == info[u].r - 1) {
                src.apply(info[u].info);
                return;
            }
            ll mid = (info[u].l + info[u].r) / 2;
            if(x < mid) {
                if(info[u].ls == -1) {
                    info[u].ls = info.size();
                    info.emplace_back(info[u].l, mid);
                }
                upd(upd, x, src, info[u].ls);
            } else {
                if(info[u].rs == -1) {
                    info[u].rs = info.size();
                    info.emplace_back(mid, info[u].r);
                }
                upd(upd, x, src, info[u].rs);
            }
            info[u].info = Info{};
            if(info[u].ls != -1)
                info[u].info = info[u].info + info[info[u].ls].info;
            if(info[u].rs != -1)
                info[u].info = info[u].info + info[info[u].rs].info;
        });
    }
    Info query(ll l, ll r) const {
        return [&](auto &&qry) {
```

```
            return qry(qry, l, r, 0);
        }([&](auto &&qry, ll ql, ll qr, ll u) -> Info {
            if(ql <= info[u].l && qr >= info[u].r) return info[u].info;
            Info ret{};
            ll mid = (info[u].l + info[u].r) / 2;
            if(ql < mid && info[u].ls != -1)
                ret = ret + qry(qry, ql, qr, info[u].ls);
            if(qr > mid && info[u].rs != -1)
                ret = ret + qry(qry, ql, qr, info[u].rs);
            return ret;
        });
    }

private:
    struct _ {
        Info info;
        ll l, r, ls, rs;
        _() : info{}, l(0), r(0), ls(-1), rs(-1) {}
        _(ll _l, ll _r) : info{}, l(_l), r(_r), ls(-1), rs(-1) {}
    };
    vector<_> info;
    ll n;
};
```

隐式 Treap

```
template<class Info, class Tag>
concept TreapInfoTag = requires(Info a, Info b, Tag c, Tag d, vector<Info> vi,
                                vector<Tag> vt, int u) {
    Info{};
    Tag{};
    { a + b } -> same_as<Info>;
    { c.apply(a, u) } -> same_as<void>;
    { c.apply(d, u) } -> same_as<void>;
    { c.empty() } -> same_as<bool>;
    { c.clear() } -> same_as<void>;
    { c.need_swap() } -> same_as<bool>;
    { vi[u] } -> same_as<Info &>;
    { vt[u] } -> same_as<Tag &>;
};

template<class Info, class Tag>
    requires(TreapInfoTag<Info, Tag>)
struct ImpTreap {
    ImpTreap() = default;
    ~ImpTreap() {
        [&](auto &&dfs) { dfs(dfs, root); }([&](auto &&dfs, Node *cur) -> void {
            if(!cur) return;
            dfs(dfs, cur->l);
            dfs(dfs, cur->r);
            delete cur;
        });
    }
    int size() const {
        return siz(root);
    }
    void push_back(const Info &info) {
        root = merge(root, new Node(info));
    }
    void insert(int pos, const Info &info) {
        auto [a, b] = split(root, pos);
        root = merge(a, merge(new Node(info), b));
    }
    void apply(int l, int r, const Tag &tag) {
        auto [a, b] = split(root, l);
        auto [c, d] = split(b, r - l);
        apply(c, tag);
        root = merge(a, merge(c, d));
    }
```

```
    Info query(int l, int r) {
        auto [a, b] = split(root, l);
        auto [c, d] = split(b, r - l);
        Info ret = c ? c->info : Info{};
        root = merge(a, merge(c, d));
        return ret;
    }

private:
    static inline mt19937 rng{
        chrono::steady_clock::now().time_since_epoch().count()};
    struct Node {
        Info self, info;
        Tag tag;
        int pri, siz;
        Node *l = nullptr, *r = nullptr;
        Node(const Info &v)
            : self(v), info(v), tag(), pri((int)rng()), siz(1) {}
    };
    Node *root = nullptr;
    static int siz(Node *p) {
        return p ? p->siz : 0;
    }
    static Info info(Node *p) {
        return p ? p->info : Info{};
    }
    static void pull(Node *p) {
        if(!p) return;
        p->siz = 1 + siz(p->l) + siz(p->r);
        p->info = info(p->l) + p->self + info(p->r);
    }
    static void apply(Node *p, const Tag &tag) {
        if(!p) return;
        if(tag.need_swap()) swap(p->l, p->r);
        tag.apply(p->self, 1);
        tag.apply(p->info, p->siz);
        tag.apply(p->tag, p->siz);
    }
    static void push(Node *p) {
        if(!p || p->tag.empty()) return;
        apply(p->l, p->tag);
        apply(p->r, p->tag);
        p->tag.clear();
    }
    static Node *merge(Node *l, Node *r) {
```

```
        if(!l) return r;
        if(!r) return l;
        if(l->pri > r->pri) {
            push(l);
            l->r = merge(l->r, r);
            pull(l);
            return l;
        }
        push(r);
        r->l = merge(l, r->l);
        pull(r);
        return r;
    }
    static pair<Node *, Node *> split(Node *p, int cnt_left) {
        if(!p) return {nullptr, nullptr};
        push(p);
        if(cnt_left <= siz(p->l)) {
            auto [a, b] = split(p->l, cnt_left);
            p->l = b;
            pull(p);
            return {a, p};
        }
        auto [a, b] = split(p->r, cnt_left - siz(p->l) - 1);
        p->r = a;
        pull(p);
        return {p, b};
    }
};
```

莫队

```cpp
struct Query {
    int idx;
    // 左闭右开
    int l, r;
    int ans;
};
// 查询区间内满足元素出现次数等于自身的数的个数
void mo_algo(vector<Query> &queries, const vector<int> &nums) {
    int n = nums.size();
    int len = sqrt(n);
    sort(queries.begin(), queries.end(), [&](const Query &ql, const Query &qr) {
        int atl = ql.l / len, atr = qr.l / len;
        if(atl != atr) return atl < atr;
        return (bool)((atl % 2 == 0) ^ (ql.r < qr.r));
    });
    int l = 0, r = 0, ans = 0;
    vector<int> count(n + 5, 0);
    auto add = [&](int x) {
        if(x >= n + 5) return;
        if(count[x] == x) --ans;
        count[x]++;
        if(count[x] == x) ++ans;
    };
    auto remove = [&](int x) {
        if(x >= n + 5) return;
        if(count[x] == x) --ans;
        count[x]--;
        if(count[x] == x) ++ans;
    };
    for(auto &q : queries) {
        // 抄板子时注意顺序，先加再更新还是先更新再加
        while(l > q.l) {
            --l;
            add(nums[l]);
        }
        while(l < q.l) {
            remove(nums[l]);
            ++l;
        }
        while(r < q.r) {
            add(nums[r]);
            ++r;
        }
```

```
        while(r > q.r) {
            --r;
            remove(nums[r]);
        }
        q.ans = ans;
    }
    sort(queries.begin(), queries.end(),
         [](const Query &l, const Query &r) { return l.idx < r.idx; });
}
```

ST表

```
template<class T, class F>
struct Sparse {
    Sparse(const vector<T> &v, F f) : func(f) {
        int n = v.size();
        int k = bit_width<unsigned>(n);
        table.resize(n);
        for(int i = 0; i < n; ++i) {
            table[i][0] = v[i];
        }
        for(int j = 1; j < k; ++j) {
            int st = 1 << (j - 1);
            for(int i = 0; i + st < n; ++i) {
                table[i][j] = func(table[i][j - 1], table[i + st][j - 1]);
            }
        }
    }
    T query(int l, int r) const {
        int len = r - l;
        int j = bit_width<unsigned>(len) - 1;
        return func(table[l][j], table[r - 1 - (1 << j) + 1][j]);
    }

private:
    vector<array<T, 25>> table;
    F func;
};
template<class T, class F>
Sparse(const vector<T> &v, F f) -> Sparse<T, F>;
```

可删堆

```cpp
template<class T, class Cmp = less<T>, class Eq = equal_to<T>>
struct DelHeap {
    DelHeap() = default;
    int size() {
        clear();
        return pq.size() - del.size();
    }
    bool empty() {
        return size() == 0;
    }
    void clear() {
        while(!pq.empty() && !del.empty() && eqf(pq.top(), del.top())) {
            pq.pop();
            del.pop();
        }
    }
    void push(T x) {
        clear();
        pq.push(x);
    }
    // 请保证 x 在这个堆里，否则可删堆会失效
    void pop(T x) {
        del.push(x);
        clear();
    }
    T top() {
        clear();
        assert(!pq.empty());
        return pq.top();
    }

private:
    priority_queue<T, vector<T>, Cmp> pq, del;
    Eq eqf;
};
```

虚树

```
struct VirtualTree {
    vector<int> tin, tout, dep, siz;
    vector<vector<int>> g, vg;
    vector<array<int, 20>> up;
    vector<int> iv;
    int ur = -1, vr = -1;
    VirtualTree(int n)
        : tin(n, -1), tout(n, -1), dep(n, -1), siz(n, -1), g(n), vg(n), up(n) {}
    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    /*
    o_开头的函数：原图中的函数，如建原树、求LCA、判断祖先关系等
    */
    void o_build(int r = 0) {
        ur = r;
        int timer = 0;
        [&](auto &&f) { f(f, r, -1); }([&](auto &&dfs, int u, int fa) -> void {
            tin[u] = timer++;
            dep[u] = fa == -1 ? 0 : dep[fa] + 1;
            up[u][0] = fa;
            for(int i = 1; i < 20; ++i) {
                up[u][i] = up[u][i - 1] == -1 ? -1 : up[up[u][i - 1]][i - 1];
            }
            siz[u] = 1;
            for(int v : g[u]) {
                if(v != fa) {
                    dfs(dfs, v, u);
                    siz[u] += siz[v];
                }
            }
            tout[u] = timer;
        });
    }
    // 注意：返回值为 u 是不是 v 的祖先
    bool o_anc(int u, int v) {
        if(u == -1) return true;
        if(v == -1) return false;
        return tin[u] <= tin[v] && tout[v] <= tout[u];
    }
    int o_lca(int u, int v) {
        if(o_anc(u, v)) return u;
```

```
        if(o_anc(v, u)) return v;
        for(int i = 19; i >= 0; --i) {
            if(!o_anc(up[u][i], v)) u = up[u][i];
        }
        return up[u][0];
    }
    /*
    v_开头的函数：虚树相关，包含清空、建虚树等
    */
    void v_clear() {
        for(int x : iv) vg[x].clear();
        iv.clear();
        vr = -1;
    }
    void v_build(vector<int> imp) {
        v_clear();
        sort(imp.begin(), imp.end(),
             [&](int x, int y) { return tin[x] < tin[y]; });
        imp.erase(unique(imp.begin(), imp.end()), imp.end());
        int n = imp.size();
        for(int i = 1; i < n; ++i) {
            imp.push_back(o_lca(imp[i - 1], imp[i]));
        }
        sort(imp.begin(), imp.end(),
             [&](int x, int y) { return tin[x] < tin[y]; });
        imp.erase(unique(imp.begin(), imp.end()), imp.end());
        vector<int> stk;
        for(int x : imp) {
            while(!stk.empty() && !o_anc(stk.back(), x)) {
                stk.pop_back();
            }
            if(!stk.empty()) {
                vg[stk.back()].push_back(x);
                vg[x].push_back(stk.back());
            } else {
                vr = x;
            }
            stk.push_back(x);
        }
        iv = move(imp);
    }
};
```

动态 BitSet

```
struct DynamicBitSet {
    explicit DynamicBitSet(int n = 0) : nums((n + 63) >> 6, 0), sz(n) {}
    void resize(int n) {
        nums.resize((n + 63) >> 6);
        sz = n;
    }
    bool getbit(int x) const {
        int u = x >> 6, v = x & 63;
        return ((nums[u] >> v) & 1) == 1;
    }
    void setbit(int x, bool val) {
        int u = x >> 6, v = x & 63;
        if(!val) {
            nums[u] &= (~(1ull << v));
        } else {
            nums[u] |= (1ull << v);
        }
    }
    void flipbit(int x) {
        setbit(x, !getbit(x));
    }
    DynamicBitSet &operator&=(const DynamicBitSet &other) {
        for(int i = 0; i < nums.size(); ++i) nums[i] &= other.nums[i];
        return *this;
    }
    DynamicBitSet operator&(const DynamicBitSet &other) const {
        return DynamicBitSet(*this) &= other;
    }
    DynamicBitSet &operator|=(const DynamicBitSet &other) {
        for(int i = 0; i < nums.size(); ++i) nums[i] |= other.nums[i];
        return *this;
    }
    DynamicBitSet operator|(const DynamicBitSet &other) const {
        return DynamicBitSet(*this) |= other;
    }
    DynamicBitSet &operator^=(const DynamicBitSet &other) {
        for(int i = 0; i < nums.size(); ++i) nums[i] ^= other.nums[i];
        return *this;
    }
    DynamicBitSet operator^(const DynamicBitSet &other) const {
        return DynamicBitSet(*this) ^= other;
    }
    DynamicBitSet &operator<<=(int z) {
```

```
    int block = z >> 6, rem = z & 63, n = nums.size();
    if(rem == 0) {
        for(int i = n - 1; i >= block; --i) nums[i] = nums[i - block];
        fill(nums.begin(), nums.begin() + min(block, n), 0);
    } else {
        ull o1 = 0, o0 = 0;
        for(int i = n - block - 1; i >= 0; --i) {
            o0 = (nums[i] << rem);
            o1 |= (nums[i] >> (64 - rem));
            if(i + block + 1 < n) nums[i + block + 1] = o1;
            o1 = o0;
        }
        fill(nums.begin(), nums.begin() + min(block, n), 0);
        if(block < n) nums[block] = o0;
    }
    return *this;
}
DynamicBitSet operator<<(int z) const {
    return DynamicBitSet(*this) <<= z;
}
DynamicBitSet &operator>>=(int z) {
    int block = z >> 6, rem = z & 63, n = nums.size();
    if(rem == 0) {
        for(int i = 0; i < n - block; ++i) nums[i] = nums[i + block];
        fill(nums.end() - min(n, block), nums.end(), 0);
    } else {
        ull o0 = 0, o1 = 0;
        for(int i = 0; i + block < n; ++i) {
            o1 |= (nums[i + block] << (64 - rem));
            o0 = (nums[i + block] >> rem);
            if(i != 0) nums[i - 1] = o1;
            o1 = o0;
        }
        fill(nums.end() - min(n, block + 1), nums.end(), 0);
        if(block <= n) nums[n - block - 1] = o0;
    }
    return *this;
}
DynamicBitSet operator>>(int z) const {
    return DynamicBitSet(*this) >>= z;
}
bool allzero() const {
    int block = sz >> 6, rem = sz & 63;
    for(int i = 0; i < block; ++i) {
        if(nums[i] != 0ull) return false;
```

```
        }
        for(int i = block << 6; i < sz; ++i) {
            if(getbit(i)) return false;
        }
        return true;
    }
    int size() const {
        return sz;
    }
    int count() const {
        if(nums.empty()) return 0;
        int ans = 0;
        for(int i = 0; i < nums.size() - 1; ++i) {
            ans += popcount(nums[i]);
        }
        for(int i = int(nums.size() * 64) - 64; i < sz; ++i) {
            ans += int(getbit(i));
        }
        return ans;
    }

private:
    vector<ull> nums;
    int sz;
};
```

图论

Dijkstra

```
template<WeightedEdgeT Edge>
vector<ll> dijkstra(const vector<vector<Edge>> &g, int s) {
    int n = g.size();
    vector<ll> di(n, inf);
    di[s] = 0;
    vector<bool> vis(n, false);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
    pq.emplace(0, s);
    while(!pq.empty()) {
        auto [_, u] = pq.top();
        pq.pop();
        if(vis[u]) continue;
        vis[u] = true;
        for(auto &e : g[u]) {
            if(!vis[e.v] && di[e.v] > di[u] + e.w) {
                di[e.v] = di[u] + e.w;
                pq.emplace(di[e.v], e.v);
            }
        }
    }
    return di;
}
```

Floyd

```
// 操作后，graph就变成了最短路
void floyd(vector<vector<ll>> &g) {
    for(int k = 0; k < g.size(); ++k)
        for(int i = 0; i < g.size(); ++i)
            for(int j = 0; j < g.size(); ++j)
                if(g[i][j] > g[i][k] + g[k][j]) g[i][j] = g[i][k] + g[k][j];
}
```

SPFA

```cpp
// 返回空vector说明有负环
template<WeightedEdgeT Edge>
vector<ll> spfa(const vector<vector<Edge>> &g, int s) {
    int n = g.size();
    vector<ll> d(n, inf);
    vector<int> cnt(n, 0);
    vector<bool> inq(n, false);
    queue<int> q;
    q.push(s);
    d[s] = 0;
    inq[s] = true;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for(auto &e : g[u]) {
            if(d[e.v] > d[u] + e.w) {
                d[e.v] = d[u] + e.w;
                if(!inq[e.v]) {
                    inq[e.v] = true;
                    q.push(e.v);
                    cnt[e.v] = cnt[u] + 1;
                    if(cnt[e.v] > n) return {};
                }
            }
        }
    }
    return d;
}
```

Kruskal

```cpp
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int u, v;
    ll w;
};

ll kruskal(vector<Edge> &es, int n) {
    DSU d(n);
    ll ans = 0;
    sort(es.begin(), es.end(),
         [](const Edge &a, const Edge &b) { return a.w < b.w; });
    for(auto &e : es) {
        if(!d.is_connected(e.u, e.v)) {
            d.connect(e.u, e.v);
            ans += e.w;
        }
    }
    for(int i = 1; i < n; ++i) {
        if(!d.is_connected(i, 0)) return inf;
    }
    return ans;
}
```

Prim

```cpp
template<WeightedEdgeT Edge>
ll prim(const vector<vector<Edge>> &g) {
    int n = g.size();
    vector<bool> vis(n, false);
    vector<ll> d(n, inf);
    d[0] = 0;
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
    pq.emplace(0, 0);
    ll ret = 0;
    while(!pq.empty()) {
        auto [w, v] = pq.top();
        pq.pop();
        if(vis[v]) continue;
        vis[v] = true;
        ret += w;
        for(auto &e : g[v]) {
            if(!vis[e.v] && d[e.v] > e.w) {
                d[e.v] = e.w;
                pq.emplace(e.w, e.v);
            }
        }
    }
    for(bool b : vis) {
        if(!b) return inf;
    }
    return ret;
}
```

Boruvka

```cpp
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int u, v;
    ll w;
};

ll boruvka(const vector<Edge> &es, int n) {
    int m = es.size();
    DSU dsu(n);
    ll ans = 0;
    int c = n;
    vector<int> mst(n);
    while(c > 1) {
        for(int i = 0; i < n; ++i) {
            if(dsu.find(i) == i) mst[i] = -1;
        }
        for(int i = 0; i < m; ++i) {
            auto [u, v, w] = es[i];
            int fu = dsu.find(u), fv = dsu.find(v);
            if(fu == fv) continue;
            if(mst[fu] == -1 || es[mst[fu]].w > w) mst[fu] = i;
            if(mst[fv] == -1 || es[mst[fv]].w > w) mst[fv] = i;
        }
        bool flag = false;
        for(int i = 0; i < n; ++i) {
            if(dsu.find(i) != i || mst[i] == -1) continue;
            auto [u, v, w] = es[mst[i]];
            if(!dsu.is_connected(u, v)) {
                dsu.connect(u, v);
                ans += w;
                --c;
                flag = true;
            }
        }
        if(!flag) return inf;
    }
    return ans;
}
```

Tarjan

```cpp
template<class T>
concept EdgeT = requires(T t) {
    { t.v } -> convertible_to<int>;
};

// Strongly connected components
struct SCC {
    explicit SCC(int n) : dfn(n, -1), low(n, -1), inscc(n, -1), ins(n), g(n) {}
    explicit SCC(const vector<vector<int>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), inscc(g.size(), -1),
          ins(g.size()), g(g) {
        build();
    }
    template<EdgeT T>
    explicit SCC(const vector<vector<T>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), inscc(g.size(), -1),
          g(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) add_edge(u, edge.v);
        }
        build();
    }
    void add_edge(int u, int v) {
        if(u != v) g[u].push_back(v);
    }
    void build() {
        int ndfn = 0, cnt = 0;
        [&](auto &&dfs) {
            for(int i = 0; i < g.size(); i++) {
                if(dfn[i] == -1) dfs(dfs, i);
            }
        }([&](auto &&dfs, int u) -> void {
            dfn[u] = low[u] = ndfn++;
            stk.push(u);
            ins[u] = true;
            for(int v : g[u]) {
                if(dfn[v] == -1) {
                    dfs(dfs, v);
                    low[u] = min(low[u], low[v]);
                } else if(ins[v]) {
                    low[u] = min(low[u], dfn[v]);
                }
```

```
            }
            if(dfn[u] == low[u]) {
                vector<int> scc;
                int v = -1;
                while(v != u) {
                    v = stk.top();
                    stk.pop();
                    ins[v] = false;
                    inscc[v] = cnt;
                    scc.emplace_back(v);
                }
                sccs.emplace_back(move(scc));
                ++cnt;
            }
        });
        dag.assign(cnt, {});
        set<pair<int, int>> edges;
        for(int u = 0; u < g.size(); ++u) {
            for(int v : g[u]) {
                int bu = inscc[u], bv = inscc[v];
                if(bu != bv && !edges.contains({bu, bv})) {
                    dag[bu].push_back(bv);
                    edges.insert({bu, bv});
                }
            }
        }
    }
    vector<int> dfn, low, inscc;
    vector<vector<int>> g, sccs, dag;

private:
    vector<bool> ins;
    stack<int> stk;
};

// Edge-biconnected components
struct EBCC {
    explicit EBCC(int n) : dfn(n, -1), low(n, -1), g(n), in_ebcc(n) {}
    explicit EBCC(const vector<vector<int>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), g(g), in_ebcc(g.size()) {
        build();
    }
    template<EdgeT T>
    explicit EBCC(const vector<vector<T>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), g(g.size()), in_ebcc(g.size()) {
```

```
    int n = g.size();
    for(int u = 0; u < n; ++u) {
        for(const T &edge : g[u]) add_edge(u, edge.v);
    }
    build();
}
void add_edge(int u, int v) {
    if(u == v) return;
    g[u].push_back(v);
    g[v].push_back(u);
}
void build() {
    int ndfn = 0;
    vector<bool> ins(dfn.size(), false);
    stack<int> stk;
    [&](auto &&dfs) {
        for(int i = 0; i < g.size(); ++i) {
            if(dfn[i] == -1) dfs(dfs, i, -1);
        }
    }([&](auto &&dfs, int u, int fa) -> void {
        dfn[u] = low[u] = ndfn++;
        ins[u] = true;
        stk.push(u);
        for(int v : g[u]) {
            if(v == fa) continue;
            if(dfn[v] == -1) {
                dfs(dfs, v, u);
                low[u] = min(low[u], low[v]);
            } else if(ins[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if(dfn[u] == low[u]) {
            vector<int> t;
            int n = -1;
            while(n != u) {
                n = stk.top();
                stk.pop();
                t.push_back(n);
                ins[n] = false;
            }
            ebcc.emplace_back(move(t));
        }
    });
    for(int i = 0; i < ebcc.size(); ++i) {
```

```
            for(int u : ebcc[i]) in_ebcc[u] = i;
        }
    }
    vector<vector<int>> g, ebcc;
    vector<int> in_ebcc, dfn, low;
};

// Vertex-biconnected components
struct DCC {
    explicit DCC(int n) : dfn(n, -1), low(n, -1), g(n) {}
    explicit DCC(const vector<vector<int>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), g(g) {
        build();
    }
    template<EdgeT T>
    explicit DCC(const vector<vector<T>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), g(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) add_edge(u, edge.v);
        }
        build();
    }
    void add_edge(int u, int v) {
        if(u == v) return;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    void build() {
        int ndfn = 0;
        stack<int> stk;
        [&](auto &&dfs) {
            for(int i = 0; i < g.size(); ++i) {
                if(dfn[i] == -1) dfs(dfs, i, -1, i);
            }
        }([&](auto &&dfs, int u, int fa, int rt) -> void {
            dfn[u] = low[u] = ndfn++;
            stk.push(u);
            int child = 0;
            for(int v : g[u]) {
                if(v == fa) continue;
                if(dfn[v] == -1) {
                    dfs(dfs, v, u, rt);
                    low[u] = min(low[u], low[v]);
                    if(low[v] >= dfn[u]) {
```

```
                        ++child;
                        vector<int> f = {u};
                        while(!stk.empty()) {
                            int x = stk.top();
                            stk.pop();
                            f.push_back(x);
                            if(x == v) break;
                        }
                        dcc.emplace_back(move(f));
                    }
                } else {
                    low[u] = min(low[u], dfn[v]);
                }
            }
            if(u == rt && g[u].empty()) {
                dcc.push_back(vector<int>{u});
                return;
            }
        });
    }
    vector<int> dfn, low;
    vector<vector<int>> g, dcc;
};

// Articulation points
struct AP {
    explicit AP(int n) : dfn(n, -1), low(n, -1), g(n) {}
    explicit AP(const vector<vector<int>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), g(g) {
        build();
    }
    template<EdgeT T>
    explicit AP(const vector<vector<T>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), g(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) add_edge(u, edge.v);
        }
        build();
    }
    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    void build() {
```

```
        int ndfn = 0;
        [&](auto &&dfs) {
            for(int i = 0; i < g.size(); ++i) {
                if(dfn[i] == -1) dfs(dfs, i, -1);
            }
        }([&](auto &&dfs, int u, int fa) -> void {
            dfn[u] = low[u] = ndfn++;
            int child = 0;
            bool flag = false;
            for(int v : g[u]) {
                if(dfn[v] == -1) {
                    ++child;
                    dfs(dfs, v, u);
                    low[u] = min(low[u], low[v]);
                    if(fa != -1) {
                        flag |= (low[v] >= dfn[u]);
                    }
                } else if(v != fa) {
                    low[u] = min(low[u], dfn[v]);
                }
            }
            if(fa == -1) flag = (child > 1);
            if(flag) ap.push_back(u);
        });
        sort(ap.begin(), ap.end());
        ap.erase(unique(ap.begin(), ap.end()), ap.end());
    }
    vector<int> dfn, low;
    vector<vector<int>> g;
    vector<int> ap;
};

// Bridges
struct Bridge {
    explicit Bridge(int n) : dfn(n, -1), low(n, -1), g(n) {}
    explicit Bridge(const vector<vector<int>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), g(g) {
        build();
    }
    template<EdgeT T>
    explicit Bridge(const vector<vector<T>> &g)
        : dfn(g.size(), -1), low(g.size(), -1), g(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) add_edge(u, edge.v);
```

```
        }
        build();
    }
    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    void build() {
        int ndfn = 0;
        [&](auto &&dfs) {
            for(int i = 0; i < g.size(); ++i) {
                if(dfn[i] == -1) dfs(dfs, i, -1);
            }
        }([&](auto &&dfs, int u, int fa) -> void {
            dfn[u] = low[u] = ndfn++;
            for(int v : g[u]) {
                if(dfn[v] == -1) {
                    dfs(dfs, v, u);
                    low[u] = min(low[u], low[v]);
                    if(low[v] > dfn[u]) {
                        bridges.emplace_back(min(u, v), max(u, v));
                    }
                } else if(v != fa) {
                    low[u] = min(low[u], dfn[v]);
                }
            }
        });
    }
    vector<int> dfn, low;
    vector<vector<int>> g;
    vector<pair<int, int>> bridges;
};
```

匈牙利算法

```cpp
int hungarian(const vector<vector<int>> &g, int vs) {
    int us = g.size();
    vector<int> mu(us, -1);
    vector<int> mv(vs, -1);
    auto dfs = [&](auto &&dfs, int u, vector<bool> &vis) -> bool {
        for(int v : g[u]) {
            if(vis[v]) continue;
            vis[v] = true;
            if(mv[v] == -1 || dfs(dfs, mv[v], vis)) {
                mv[v] = u;
                mu[u] = v;
                return true;
            }
        }
        return false;
    };
    int ret = 0;
    for(int u = 0; u < us; ++u) {
        if(mu[u] == -1) {
            vector<bool> vis(vs, false);
            if(dfs(dfs, u, vis)) ret++;
        }
    }
    return ret;
}
```

Dinic

```
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;

struct Dinic {
    struct Edge {
        int v, r;
        ll w;
    };
    vector<vector<Edge>> g;
    Dinic(int n, int s = -1, int t = -1) : g(n), lv(n), _s(s), _t(t), n(n) {}
    void add_edge(int u, int v, ll w) {
        int iv = g[v].size(), iu = g[u].size();
        g[u].push_back({v, iv, w});
        g[v].push_back({u, iu, 0});
    }
    ll maxflow(int s = -1, int t = -1) {
        if(_s == -1) _s = s;
        if(_t == -1) _t = t;
        ll mf = 0;
        auto dfs = [&](auto &&dfs, int u, ll mf) -> ll {
            if(u == _t || mf == 0) return mf;
            ll nf = 0;
            for(auto &e : g[u]) {
                if(lv[e.v] == lv[u] + 1 && e.w > 0) {
                    ll min_flow = min(mf, e.w);
                    ll push = dfs(dfs, e.v, min_flow);
                    if(push > 0) {
                        e.w -= push;
                        g[e.v][e.r].w += push;
                        nf += push;
                        mf -= push;
                        if(mf == 0) return nf;
                    }
                }
            }
            return nf;
        };
        while(bfs()) {
            ll flow = dfs(dfs, _s, inf);
            while(flow > 0) {
                mf += flow;
                flow = dfs(dfs, _s, inf);
            }
        }
```

```
        return mf;
    }

private:
    vector<int> lv;
    int n, _s, _t;
    bool bfs() {
        fill(lv.begin(), lv.end(), -1);
        lv[_s] = 0;
        queue<int> q;
        q.emplace(_s);
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            for(const auto &e : g[u]) {
                if(lv[e.v] == -1 && e.w > 0) {
                    lv[e.v] = lv[u] + 1;
                    q.emplace(e.v);
                }
            }
        }
        return lv[_t] != -1;
    }
};
```

费用流

```cpp
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;

struct MCMF {
    struct Edge {
        int v, r;
        ll w, c;
    };
    int n;
    vector<vector<Edge>> g;
    explicit MCMF(int _n) : n(_n), g(_n) {}
    void add_edge(int u, int v, ll f, ll w) {
        int iu = g[u].size(), iv = g[v].size();
        g[u].push_back({v, iv, w, f});
        g[v].push_back({u, iu, -w, 0});
    }
    pair<ll, ll> mcmf(int s, int t) {
        ll mc = 0, mf = 0;
        vector<ll> h(n, inf);
        h[s] = 0;
        for(int k = 1; k < n; ++k) {
            bool cg = false;
            for(int u = 0; u < n; ++u) {
                if(h[u] == inf) continue;
                for(auto &e : g[u]) {
                    if(e.c > 0 && h[e.v] > h[u] + e.w) {
                        h[e.v] = h[u] + e.w;
                        cg = true;
                    }
                }
            }
            if(!cg) break;
        }
        for(ll &x : h) {
            if(x == inf) x = 0;
        }
        while(true) {
            vector<int> pv(n, -1), pe(n, -1);
            vector<ll> d(n, inf);
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
            d[s] = 0;
            pq.emplace(0, s);
            while(!pq.empty()) {
                auto [du, u] = pq.top();
```

```
            pq.pop();
            if(du != d[u]) continue;
            for(int i = 0; i < g[u].size(); ++i) {
                const auto &e = g[u][i];
                if(e.c <= 0) continue;
                ll nd = du + e.w + h[u] - h[e.v];
                if(nd < d[e.v]) {
                    d[e.v] = nd;
                    pv[e.v] = u;
                    pe[e.v] = i;
                    pq.emplace(nd, e.v);
                }
            }
        }
        if(d[t] == inf) break;
        for(int v = 0; v < n; ++v) {
            if(d[v] != inf) h[v] += d[v];
        }
        ll add = inf;
        for(int v = t; v != s; v = pv[v]) {
            chkmin(add, g[pv[v]][pe[v]].c);
        }
        for(int v = t; v != s; v = pv[v]) {
            int u = pv[v], i = pe[v];
            int r = g[u][i].r;
            g[u][i].c -= add;
            g[v][r].c += add;
        }
        mf += add;
        mc += add * (h[t] - h[s]);
    }
    return {mc, mf};
  }
};
```

动态规划

背包DP

```
struct Item {
    ll w, v;
};

ll knapsack_01(const vector<Item> &s, int mw) {
    vector<ll> dp(mw + 1, 0);
    for(auto [w, v] : s) {
        for(int ww = mw; ww >= w; --ww) {
            chkmax(dp[ww], dp[ww - w] + v);
        }
    }
    return dp[mw];
}
ll knapsack_full(const vector<Item> &s, int mw) {
    vector<ll> dp(mw + 1, 0);
    for(auto [w, v] : s) {
        for(int ww = w; ww <= mw; ++ww) {
            chkmax(dp[ww], dp[ww - w] + v);
        }
    }
    return dp[mw];
}

struct MultiItem {
    ll w, v, cnt;
};

ll knapsack_multi(const vector<MultiItem> &s, int mw) {
    vector<Item> ss;
    for(auto [w, v, cnt] : s) {
        ll i = 1;
        while(i < cnt) {
            ss.push_back({w * i, v * i});
            cnt -= i;
            i *= 2;
        }
        ss.push_back({w * cnt, v * cnt});
    }
```

```
    return knapsack_01(ss, mw);
}
```

区间DP

```
ll range_dp(const vector<ll> &nums) {
    int n = nums.size();
    vector<ll> a(nums);
    a.insert(a.end(), nums.begin(), nums.end());
    vector<ll> sum(2 * n + 1);
    for (int i = 0; i < 2 * n; ++i) sum[i + 1] = sum[i] + a[i];
    vector<vector<ll>> dp(2 * n + 1, vector<ll>(2 * n + 1, 0));
    for (int len = 2; len <= n; ++len) {
        for (int i = 0; i + len <= 2 * n; ++i) {
            int j = i + len;
            ll best = 0;
            for (int k = i + 1; k < j; ++k) {
                best = max(best, dp[i][k] + dp[k][j]);
            }
            dp[i][j] = best + sum[j] - sum[i];
        }
    }
    ll ans = 0;
    for (int i = 0; i < n; ++i) ans = max(ans, dp[i][i + n]);
    return ans;
}
```

数位DP

```
ll digit_dp(const string &k, int d) {
    constexpr ll modulo = 1'000'000'007;
    vector<vector<ll>> memo(k.size(), vector<ll>(d, -1));
    auto dfs = [&](auto &&dfs, int idx, int psum, bool isnum, bool islimit) -> ll {
        if (idx == (int)k.size()) return (ll)(isnum && psum == 0);
        if (isnum && !islimit && memo[idx][psum] != -1) return memo[idx][psum];
        ll ans = 0;
        if (!isnum) ans = (ans + dfs(dfs, idx + 1, 0, false, false)) % modulo;
        int llim = isnum ? 0 : 1;
        int hlim = islimit ? (k[idx] - '0') : 9;
        for (int x = llim; x <= hlim; ++x) {
            ans = (ans + dfs(dfs, idx + 1, (psum + x) % d, true, islimit && x == hlim)) % modulo;
        }
        if (isnum && !islimit) memo[idx][psum] = ans;
        return ans;
    };
    return dfs(dfs, 0, 0, false, true);
}
```

数学

数学基础

```
template<ll modulo>
constexpr ll norm(ll x) {
    x %= modulo;
    if(x < 0) x += modulo;
    return x;
}

template<ll modulo>
constexpr ll qpow(ll x, ll n) {
    x = norm<modulo>(x);
    ll ret = 1;
    for(; n != 0; n >>= 1, x = x * x % modulo) {
        if(n & 1) ret = ret * x % modulo;
    }
    return ret;
}

constexpr ll ipow(ll x, ll n) {
    ll ret = 1;
    for(; n != 0; n >>= 1, x *= x) {
        if(n & 1) ret *= x;
    }
    return ret;
}
```

组合数

```cpp
template<ll modulo, ll maxn = 500005>
struct Comb {
    Comb() = delete;
    static ll fact(ll n) {
        init();
        return fac[n];
    }
    static ll invfact(ll n) {
        init();
        return ifac[n];
    }
    static ll perm(ll n, ll m) {
        init();
        if(m < 0 || m > n || n < 0) return 0;
        ll fn = fac[n], inm = ifac[n - m];
        return fn * inm % modulo;
    }
    static ll binom(ll n, ll m) {
        init();
        if(m < 0 || m > n || n < 0) return 0;
        ll fn = fac[n], im = ifac[m], inm = ifac[n - m];
        return fn * im % modulo * inm % modulo;
    }

private:
    static inline int fac[maxn], ifac[maxn];
    static inline bool inited = false;
    static void init() {
        if(inited) return;
        inited = true;
        fac[0] = 1;
        for(ll i = 1; i < maxn; ++i) {
            fac[i] = (ll(fac[i - 1]) * i) % modulo;
        }
        ifac[maxn - 1] = qpow<modulo>(fac[maxn - 1], modulo - 2);
        for(ll i = maxn - 2; i >= 0; --i) {
            ifac[i] = (ll(ifac[i + 1]) * (i + 1)) % modulo;
        }
    }
};
```

Lucas定理

```
template<ll modulo>
struct Lucas {
    Lucas() = delete;
    static ll binom(ll n, ll m) {
        init();
        if(m < 0 || m > n) return 0;
        if(m == 0) return 1;
        auto small = [&](int n, int m) {
            if(m < 0 || m > n) return 0ll;
            return ll(fact[n]) * ll(invfact[m]) % modulo * ll(invfact[n - m]) %
                   modulo;
        };
        return binom(n / modulo, m / modulo) * small(n % modulo, m % modulo) %
               modulo;
    }

private:
    static inline int fact[modulo], invfact[modulo];
    static inline bool inited = false;
    static void init() {
        if(inited) return;
        inited = true;
        fact[0] = 1;
        for(ll i = 1; i < modulo; ++i) {
            fact[i] = ll(fact[i - 1]) * i % modulo;
        }
        invfact[modulo - 1] = modulo - 1;
        for(ll i = modulo - 2; i >= 0; --i) {
            invfact[i] = ll(invfact[i + 1]) * (i + 1) % modulo;
        }
    }
};
```

线性筛

```
template<class F>
concept eularsieve_func = requires(F &&f, int p, int k, int pk) {
    // f(p, k, pk) returns f(p^k), where pk == p^k.
    { f(p, k, pk) } -> convertible_to<ll>;
};

struct EularSieve {
    vector<int> primes, lpf, lpow, lpf_pow;
    vector<ll> fv;
    explicit EularSieve(int n) : lpf(n + 1), lpow(n + 1), lpf_pow(n + 1) {
        for(int i = 2; i <= n; ++i) {
            if(lpf[i] == 0) {
                lpf[i] = i;
                lpow[i] = 1;
                lpf_pow[i] = i;
                primes.push_back(i);
            }
            for(int p : primes) {
                ll product = 1ll * i * p;
                if(product > n) break;
                int j = static_cast<int>(product);
                lpf[j] = p;
                if(i % p == 0) {
                    lpow[j] = lpow[i] + 1;
                    lpf_pow[j] = lpf_pow[i] * p;
                    break;
                } else {
                    lpow[j] = 1;
                    lpf_pow[j] = p;
                }
            }
        }
    }
    template<eularsieve_func F>
    EularSieve(int n, F &&f)
        : lpf(n + 1), lpow(n + 1), lpf_pow(n + 1), fv(n + 1) {
        if(n >= 1) fv[1] = 1;
        for(int i = 2; i <= n; ++i) {
            if(lpf[i] == 0) {
                lpf[i] = i;
                lpow[i] = 1;
                lpf_pow[i] = i;
                primes.push_back(i);
```

```
                fv[i] = f(i, 1, i);
            }
            for(int p : primes) {
                ll product = 1ll * i * p;
                if(product > n) break;
                int j = static_cast<int>(product);
                lpf[j] = p;
                if(i % p == 0) {
                    lpow[j] = lpow[i] + 1;
                    lpf_pow[j] = lpf_pow[i] * p;
                    int rem = j / lpf_pow[j];
                    fv[j] = fv[rem] * f(p, lpow[j], lpf_pow[j]);
                    break;
                } else {
                    fv[j] = fv[i] * fv[p];
                    lpow[j] = 1;
                    lpf_pow[j] = p;
                }
            }
        }
    }
};

constexpr ll eular_f(int p, int, int pk) {
    return pk - pk / p;
}
constexpr ll mobius_f(int, int k, int) {
    return k == 0 ? 1 : k == 1 ? -1 : 0;
}
constexpr ll factor_cnt_f(int, int k, int) {
    return k + 1;
}
constexpr ll factor_sum_f(int p, int, int pk) {
    return pk + (pk - 1ll) / (p - 1);
}
```

矩阵快速幂

```
template<class T>
vector<vector<T>> matmul(const vector<vector<T>> &a,
                         const vector<vector<T>> &b) {
    int m = a.size(), n = a[0].size(), p = b[0].size();
    vector<vector<T>> ret(m, vector<T>(p, T(0)));
    for(int i = 0; i < m; ++i) {
        for(int j = 0; j < p; ++j) {
            for(int k = 0; k < n; ++k) {
                ret[i][j] = ret[i][j] + a[i][k] * b[k][j];
            }
        }
    }
    return ret;
}
template<class T>
vector<vector<T>> matpow(vector<vector<T>> mat, ll n) {
    int M = mat.size();
    vector<vector<T>> ret(M, vector<T>(M, T(0)));
    for(int i = 0; i < M; ++i) {
        ret[i][i] = T(1);
    }
    for(; n != 0; n >>= 1, mat = matmul(mat, mat)) {
        if(n & 1ll) ret = matmul(ret, mat);
    }
    return ret;
}
```

矩阵类

```
template<class T>
struct Matrix {
    template<class>
    friend struct Matrix;

    Matrix() : n(0), m(0) {}
    Matrix(int _n, int _m) : x(_n * _m), n(_n), m(_m) {}
    template<class U>
    Matrix(const Matrix<U> &o) : x(o.x.begin(), o.x.end()), n(o.n), m(o.m) {}
    static Matrix unit(int _n) {
        Matrix ret(_n, _n);
        for(int i = 0; i < _n; ++i) ret[i][i] = T(1);
        return ret;
    }
    span<T> operator[](int i) {
        return span<T>(x.data() + i * m, m);
    }
    span<const T> operator[](int i) const {
        return span<const T>(x.data() + i * m, m);
    }
    Matrix &operator+=(const Matrix &r) {
        for(int i = 0; i < n * m; ++i) x[i] = x[i] + r.x[i];
        return *this;
    }
    Matrix &operator-=(const Matrix &r) {
        for(int i = 0; i < n * m; ++i) x[i] = x[i] - r.x[i];
        return *this;
    }
    Matrix &operator*=(const Matrix &r) {
        *this = *this * r;
        return *this;
    }
    Matrix pow(ll n) const {
        Matrix ret = Matrix::unit(), now = *this;
        for(; n; n >>= 1, now = now * now)
            if(n & 1) ret = ret * now;
        return ret;
    }
    friend Matrix operator+(const Matrix &l, const Matrix &r) {
        return Matrix(l) += r;
    }
    friend Matrix operator-(const Matrix &l, const Matrix &r) {
        return Matrix(l) -= r;
```

```
    }
    friend Matrix operator*(const Matrix &l, const Matrix &r) {
        int rows = l.n, mid = l.m, cols = r.m;
        Matrix ret(rows, cols);
        for(int i = 0; i < rows; ++i) {
            for(int j = 0; j < cols; ++j) {
                for(int k = 0; k < mid; ++k) {
                    ret[i][j] = ret[i][j] + l[i][k] * r[k][j];
                }
            }
        }
        return ret;
    }

private:
    vector<T> x;
    int n, m;
};
```

EXGCD

```
// @returns (gcd, x, y) so that gcd = ax + by
template<class T>
tuple<T, T, T> exgcd(T a, T b) {
    if(b == 0) return tuple(a, 1, 0);
    auto [g, x, y] = exgcd(b, a % b);
    return tuple(g, y, x - (a / b) * y);
}
```

中国剩余定理

```cpp
// @returns (a, b) so that answer is a + kb, k\in N_+
template<class T>
pair<T, T> crt(const vector<T> &rem, const vector<T> &mod) {
    int n = rem.size();
    T modulo_ = 1, ans = 0;
    for(int i = 0; i < n; ++i) {
        modulo_ *= mod[i];
    }
    for(int i = 0; i < n; ++i) {
        T m = modulo_ / mod[i];
        auto [_, b, __] = exgcd<T>(m, mod[i]);
        ans = (ans + ((rem[i] * m) % modulo_ * b) % modulo_) % modulo_;
    }
    return pair((ans % modulo_ + modulo_) % modulo_, modulo_);
}

template<class T>
pair<T, T> excrt(const vector<T> &rem, const vector<T> &mod) {
    int n = rem.size();
    T r1 = rem[0], m1 = mod[0];
    for(int i = 1; i < n; ++i) {
        T r2 = rem[i], m2 = mod[i];
        auto [g, p, _] = exgcd<T>(m1, m2);
        if((r2 - r1) % g != 0) {
            return {-1, -1};
        }
        T v = m2 / g, x = (r2 - r1) / g;
        T u = p % v * x % v;
        T w = (u % v + v) % v;
        r1 += w * m1;
        m1 = lcm(m1, m2);
    }
    return {(r1 % m1 + m1) % m1, m1};
}
```

高斯消元

```
template<unsigned M>
optional<vector<vector<ModInt<M>>>> gauss(const vector<vector<ModInt<M>>> &a,
                                         const vector<vector<ModInt<M>>> &b) {
    using Z = ModInt<M>;
    int r = a.size(), n = a[0].size(), m = b[0].size(), row = 0;
    vector<vector<Z>> aug(r, vector<Z>(n + m));
    for(int i = 0; i < r; ++i) {
        for(int j = 0; j < m + n; ++j) {
            aug[i][j] = (j < n) ? a[i][j] : b[i][j - n];
        }
    }
    vector<int> where(n, -1);
    for(int col = 0; col < n && row < r; ++col) {
        int sel = row;
        while(sel < r && aug[sel][col] == Z(0)) ++sel;
        if(sel == r) continue;
        swap(aug[row], aug[sel]);
        Z inv = aug[row][col].inv();
        for(int j = col; j < n + m; ++j) {
            aug[row][j] *= inv;
        }
        for(int i = 0; i < r; ++i) {
            if(i != row && aug[i][col] != Z(0)) {
                Z f = aug[i][col];
                for(int j = col; j < n + m; ++j) {
                    aug[i][j] -= f * aug[row][j];
                }
            }
        }
        where[col] = row;
        ++row;
    }
    for(int i = row; i < r; ++i) {
        bool flag = true;
        for(int j = 0; j < n; ++j) {
            if(aug[i][j] != Z(0)) {
                flag = false;
                break;
            }
        }
        if(flag) {
            for(int j = 0; j < m; ++j) {
                if(aug[i][n + j] != Z(0)) return nullopt;
```

```
            }
        }
    }
    for(int col = 0; col < n; ++col) {
        if(where[col] == -1) return nullopt;
    }
    vector<vector<Z>> ret(n, vector<Z>(m));
    for(int col = 0; col < n; ++col) {
        for(int j = 0; j < m; ++j) {
            ret[col][j] = aug[where[col]][n + j];
        }
    }
    return ret;
}

template<unsigned M>
ModInt<M> det(vector<vector<ModInt<M>>> &mat) {
    using Z = ModInt<M>;
    int n = mat.size();
    Z ans = Z(1);
    for(int row = 0; row < n; ++row) {
        int sel = row;
        while(sel < n && mat[sel][row] == Z(0)) ++sel;
        if(sel == n) return Z(0);
        if(sel != row) {
            swap(mat[sel], mat[row]);
            ans = Z(0) - ans;
        }
        Z inv = mat[row][row].inv();
        ans *= mat[row][row];
        for(int j = row + 1; j < n; ++j) {
            Z delta = mat[j][row] * inv;
            for(int k = row; k < n; ++k) {
                mat[j][k] -= delta * mat[row][k];
            }
        }
    }
    return ans;
}
```

斯特林数

```cpp
template<ll maxn, ll modulo>
struct Sterling {
    Sterling() = delete;
    static int get(ll n, ll k) {
        init();
        if(n < 0 || k < 0 || k > n || n >= maxn) return 0;
        return ster[n][k];
    }

private:
    // static constexpr int maxn = 5005;
    static inline int ster[maxn][maxn];
    static void init() {
        static bool inited = false;
        if(inited) return;
        ster[0][0] = 1;
        for(ll i = 1; i < maxn; ++i) {
            ster[i][0] = 0;
            for(ll j = 1; j < i; ++j) {
                ster[i][j] =
                    (ll(ster[i - 1][j - 1]) + ll(ster[i - 1][j]) * j) % modulo;
            }
            ster[i][i] = 1;
        }
        inited = true;
    }
};
```

整除分块

```cpp
// 要计算\sum floor(n/i)
template<ll modulo>
ll intdiv(ll n) {
    ll ans = 0;
    for(ll b = 1; b <= n;) {
        ll val = n / b;
        ll r = n / val;
        ll len = r - b + 1;
        ans = (ans + (len % modulo * val % modulo)) % modulo;
        b = r + 1;
    }
    return ans;
}
```

线性基

```
struct LinearBasis {
    LinearBasis() : base{} {}
    void insert(ll val) {
        for(int i = 60; i >= 0; --i) {
            if(((val >> i) & 1) == 0) continue;
            if(base[i] == 0) {
                base[i] = val;
                return;
            }
            val ^= base[i];
        }
    }
    ll max_xor() const {
        ll ans = 0;
        for(int i = 60; i >= 0; --i) {
            if((ans ^ base[i]) > ans) {
                ans ^= base[i];
            }
        }
        return ans;
    }
    ll min_xor() const {
        for(int i = 0; i <= 60; ++i) {
            if(base[i] != 0) {
                return base[i];
            }
        }
        return 0;
    }
    ll kth_xor(ll k) const {
        // placeholder
        return 0;
    }
    bool can_repr(ll dest) const {
        for(int i = 60; i >= 0; --i) {
            if((dest >> i) & 1) {
                dest ^= base[i];
            }
        }
        return dest == 0;
    }

private:
```

```
    array<ll, 64> base;
};
```

模整数类

```
template<unsigned M>
struct ModInt {
    static constexpr unsigned modulo = M;
    ModInt() : val(0) {}
    ModInt(ll x) : val(norm<modulo>(x)) {}
    ModInt &operator+=(const ModInt &z) {
        val += z.val;
        if(val >= modulo) val -= modulo;
        return *this;
    }
    ModInt &operator-=(const ModInt &z) {
        if(val < z.val) val += modulo;
        val -= z.val;
        return *this;
    }
    ModInt &operator*=(const ModInt &z) {
        val = (ll)val * z.val % modulo;
        return *this;
    }
    ModInt &to_inv() {
        val = qpow<modulo>(val, modulo - 2);
        return *this;
    }
    ModInt inv() const {
        ModInt z = *this;
        return z.to_inv();
    }
    ModInt operator+(const ModInt &b) const {
        return ModInt(val) += b;
    }
    ModInt operator-(const ModInt &b) const {
        return ModInt(val) -= b;
    }
    ModInt operator*(const ModInt &b) const {
        return ModInt(val) *= b;
    }
    bool operator==(const ModInt &) const = default;
    operator unsigned() const {
        return val;
    }

private:
```

```
    unsigned val;
};
```

FFT

```
constexpr ld pi = numbers::pi_v<ld>;

using cd = complex<ld>;
vector<ld> multiply(const vector<ld> &a, const vector<ld> &b) {
    int n = 1;
    while(n < (a.size() + b.size())) {
        n <<= 1;
    }
    vector<cd> fa(n), fb(n);
    for(int i = 0; i < a.size(); ++i) {
        fa[i] = a[i];
    }
    for(int i = 0; i < b.size(); ++i) {
        fb[i] = b[i];
    }
    auto fft = [](vector<cd> &f, bool inv) -> void {
        int n = f.size();
        for(int i = 1, j = 0; i < n; ++i) {
            int bit = n >> 1;
            for(; (j & bit) != 0; bit >>= 1) j ^= bit;
            j ^= bit;
            if(i < j) swap(f[i], f[j]);
        }
        for(int len = 2; len <= n; len *= 2) {
            ld alp = 2 * pi / len;
            cd wn(cos(alp), sin(alp));
            if(inv) wn.imag(-wn.imag());
            for(int i = 0; i < n; i += len) {
                cd w = 1.0l;
                for(int j = 0; j < len / 2; ++j) {
                    cd u = f[i + j], v = f[i + j + len / 2] * w;
                    f[i + j] = u + v;
                    f[i + j + len / 2] = u - v;
                    w *= wn;
                }
            }
        }
    };
    fft(fa, false);
    fft(fb, false);
    for(int i = 0; i < n; ++i) {
        fa[i] *= fb[i];
    }
```

```
    fft(fa, true);
    for(int i = 0; i < n; ++i) {
        fa[i] /= n;
    }
    vector<ld> ret(n);
    for(int i = 0; i < n; ++i) {
        ret[i] = fa[i].real();
    }
    while(ret.size() > (a.size() + b.size() - 1)) {
        ret.pop_back();
    }
    return ret;
}
```

NTT

```cpp
template<unsigned X>
constexpr unsigned g = 0;
template<>
constexpr unsigned g<998244353> = 3;
template<>
constexpr unsigned g<469762049> = 3;
template<>
constexpr unsigned g<167772161> = 3;
template<>
constexpr unsigned g<104857601> = 3;
template<>
constexpr unsigned g<1004535809> = 3;
template<>
constexpr unsigned g<754974721> = 11;

template<unsigned X>
    requires(g<X> != 0)
vector<ModInt<X>> multiply(const vector<ModInt<X>> &a,
                           const vector<ModInt<X>> &b) {
    using Z = ModInt<X>;
    int n = bit_ceil(a.size() + b.size());
    vector<Z> ca(n), cb(n);
    for(int i = 0; i < a.size(); ++i) ca[i] = a[i];
    for(int i = 0; i < b.size(); ++i) cb[i] = b[i];
    auto ntt = [](vector<Z> &f, bool inv) -> void {
        int n = f.size();
        for(int i = 1, j = 0; i < n; ++i) {
            int bit = n >> 1;
            for(; (j & bit) != 0; bit >>= 1) j ^= bit;
            j ^= bit;
            if(i < j) swap(f[i], f[j]);
        }
        for(int len = 2; len <= n; len *= 2) {
            Z wn = Z(qpow<X>(g<X>, (X - 1) / len));
            if(inv) wn.to_inv();
            for(int i = 0; i < n; i += len) {
                Z w = 1;
                for(int j = 0; j < len / 2; ++j) {
                    Z u = f[i + j], v = f[i + j + len / 2] * w;
                    f[i + j] = u + v;
                    f[i + j + len / 2] = u - v;
                    w *= wn;
                }
```

```
            }
        }
    };
    ntt(ca, false);
    ntt(cb, false);
    for(int i = 0; i < n; ++i) ca[i] *= cb[i];
    ntt(ca, true);
    Z in = Z(n).inv();
    for(int i = 0; i < n; ++i) ca[i] *= in;
    while(ca.size() > (a.size() + b.size() - 1)) ca.pop_back();
    return ca;
}
```

FWT

```cpp
// constexpr ll modulo = 998244353, inv2 = 499122177;
template<ll modulo>
void fwt_or(vector<ll> &a, bool invert) {
    ll n = a.size(), type = invert ? -1 : 1;
    for(ll x = 2; x <= n; x <<= 1) {
        ll k = x >> 1;
        for(ll i = 0; i < n; i += x)
            for(ll j = 0; j < k; ++j)
                a[i + j + k] = (a[i + j + k] + a[i + j] * type) % modulo;
    }
}
template<ll modulo>
void fwt_and(vector<ll> &a, bool invert) {
    ll n = a.size(), type = invert ? -1 : 1;
    for(ll x = 2; x <= n; x <<= 1) {
        ll k = x >> 1;
        for(ll i = 0; i < n; i += x)
            for(ll j = 0; j < k; ++j)
                a[i + j] = (a[i + j] + a[i + j + k] * type) % modulo;
    }
}
template<ll modulo>
void fwt_xor(vector<ll> &a, bool invert) {
    constexpr ll inv2 = qpow<modulo>(2, modulo - 2);
    ll n = a.size(), type = invert ? inv2 : 1;
    for(ll x = 2; x <= n; x <<= 1) {
        ll k = x >> 1;
        for(ll i = 0; i < n; i += x) {
            for(ll j = 0; j < k; ++j) {
                ll u = a[i + j], v = a[i + j + k];
                a[i + j] = (u + v) % modulo;
                a[i + j + k] = (u - v) % modulo;
                a[i + j] = (a[i + j] * type) % modulo;
                a[i + j + k] = (a[i + j + k] * type) % modulo;
            }
        }
    }
}
template<ll modulo>
void fwt_xnor(vector<ll> &a, bool invert) {
    reverse(a.begin(), a.end());
    fwt_xor<modulo>(a, invert);
    reverse(a.begin(), a.end());
```

```
}
template<ll modulo>
vector<ll> fwt_transform(const vector<ll> &a, const vector<ll> &b,
                         void (*func)(vector<ll> &a, bool invert)) {
    ll n = 1;
    while(n < max(a.size(), b.size())) n <<= 1;
    vector<ll> A(n, 0), B(n, 0), C(n, 0);
    for(ll i = 0; i < a.size(); i++) A[i] = a[i];
    for(ll i = 0; i < b.size(); i++) B[i] = b[i];
    func(A, false);
    func(B, false);
    for(ll i = 0; i < n; i++) {
        C[i] = (A[i] * B[i]) % modulo;
    }
    func(C, true);
    for(ll i = 0; i < n; i++) {
        C[i] = (C[i] % modulo + modulo) % modulo;
    }
    return C;
}
```

多项式

```
template<unsigned X>
    requires(g<X> != 0)
struct Polynomial {
    using Z = ModInt<X>;
    Polynomial() {}
    template<class U>
    Polynomial(const vector<U> &val) : a(val.begin(), val.end()) {}
    template<class U>
    Polynomial(initializer_list<U> val) : a(val.begin(), val.end()) {}
    int size() const {
        return a.size();
    }
    bool empty() const {
        return a.empty();
    }
    Z &operator[](int idx) {
        return a[idx];
    }
    Z operator[](int idx) const {
        return a[idx];
    }
    Polynomial &to_mod_xk(int k) {
        if(k <= 0) {
            a.clear();
        } else if(a.size() > k) {
            a.resize(k);
        }
        return *this;
    }
    Polynomial mod_xk(int k) const {
        auto b = *this;
        return b.to_mod_xk(k);
    }
    Polynomial &to_deriv() {
        if(a.empty()) return *this;
        for(int i = 1; i < a.size(); ++i) {
            a[i - 1] = a[i] * Z(i);
        }
        a.pop_back();
        return *this;
    }
    Polynomial deriv() const {
        auto b = *this;
```

```
    return b.to_deriv();
}
Polynomial &to_integ() {
    a.push_back(Z(0));
    for(int i = a.size() - 1; i > 0; --i) {
        a[i] = a[i - 1] * Z(i).inv();
    }
    a[0] = 0;
    return *this;
}
Polynomial integ() const {
    auto b = *this;
    return b.to_integ();
}
Polynomial inv(int m) const {
    Polynomial x{a[0].inv()};
    int k = 1;
    while(k < m) {
        k *= 2;
        x = x * (Polynomial{2} - mod_xk(k) * x);
        x.to_mod_xk(k);
    }
    return x.to_mod_xk(m);
}
Polynomial ln(int m) const {
    return (deriv() * inv(m)).to_integ().to_mod_xk(m);
}
Polynomial exp(int m) const {
    Polynomial x{1};
    int k = 1;
    while(k < m) {
        k *= 2;
        x *= Polynomial{1} - x.ln(k) + mod_xk(k);
        x.to_mod_xk(k);
    }
    return x.to_mod_xk(m);
}
Polynomial sqrt(int m) const {
    Polynomial x{1};
    int k = 1;
    while(k < m) {
        k *= 2;
        x = (x + mod_xk(k) * x.inv(k));
        x.to_mod_xk(k);
        x *= (X + 1) / 2;
```

```
    }
    return x.to_mod_xk(m);
}
Polynomial &operator+=(const Polynomial &b) {
    if(a.size() < b.a.size()) {
        a.resize(b.a.size(), Z{});
    }
    for(int i = 0; i < b.a.size(); ++i) {
        a[i] += b.a[i];
    }
    return *this;
}
Polynomial &operator-=(const Polynomial &b) {
    if(a.size() < b.a.size()) {
        a.resize(b.a.size(), Z{});
    }
    for(int i = 0; i < b.a.size(); ++i) {
        a[i] -= b.a[i];
    }
    return *this;
}
Polynomial &operator*=(const Polynomial &b) {
    a = multiply(a, b.a);
    return *this;
}
Polynomial &operator*=(Z k) {
    for(Z &i : a) i *= k;
    return *this;
}
Polynomial operator+(const Polynomial &b) const {
    auto c = *this;
    return c += b;
}
Polynomial operator-(const Polynomial &b) const {
    auto c = *this;
    return c -= b;
}
Polynomial operator*(const Polynomial &b) const {
    return Polynomial(multiply(a, b.a));
}
Polynomial operator*(Z k) const {
    auto c = *this;
    return c *= k;
}
```

```
private:
    vector<Z> a;
};
```

多项式插值

```cpp
template<ll modulo>
ll lagrange(const vector<ll> &x, const vector<ll> &y, ll x0) {
    int n = x.size();
    vector<ll> iden(n);
    for(int i = 0; i < n; ++i) {
        ll den = 1;
        for(int j = 0; j < n; ++j) {
            if(i != j) den = den * (x[i] - x[j] + modulo) % modulo;
        }
        iden[i] = qpow<modulo>(den, modulo - 2);
    }
    ll ret = 0;
    for(int i = 0; i < n; ++i) {
        ll num = 1;
        for(int j = 0; j < n; ++j) {
            if(i != j) num = num * (x0 - x[j] + modulo) % modulo;
        }
        ret = (ret + y[i] * num % modulo * iden[i]) % modulo;
    }
    return ret;
}

// if x[i] = i for i = 0 to x.size() - 1, then call this to solve in O(n)
template<ll modulo>
ll lagrange(const vector<ll> &y, ll x0) {
    int n = y.size();
    vector<ll> pre(n), suf(n);
    pre[0] = (x0 - 0 + modulo) % modulo;
    for(int i = 1; i < n; ++i) {
        pre[i] = pre[i - 1] * (x0 - i + modulo) % modulo;
    }
    suf[n - 1] = (x0 - (n - 1) + modulo) % modulo;
    for(int i = n - 2; i >= 0; --i) {
        suf[i] = suf[i + 1] * (x0 - i + modulo) % modulo;
    }
    ll ret = 0;
    for(int i = 0; i < n; ++i) {
        ll term = y[i];
        if(i > 0) term = term * pre[i - 1] % modulo;
        if(i < n - 1) term = term * suf[i + 1] % modulo;
        ll den = Comb<modulo>::invfact(i) * Comb<modulo>::invfact(n - 1 - i) %
                 modulo;
        if((n - 1 - i) % 2) den = modulo - den;
```

```
        ret = (ret + term * den) % modulo;
    }
    return ret;
}
```

离散对数

```
// returns ret so that qpow(ret, a) = b, -1 if not exist
template<ll modulo>
ll mlog(ll a, ll b) {
    if(b == 1) return 0;
    ll t = ceil(sqrt(modulo));
    ll now = b;
    unordered_map<ll, ll, MyHash> table;
    for(int i = 0; i < t; ++i) {
        table[now] = i;
        now = now * a % modulo;
    }
    ll mi = qpow<modulo>(a, t);
    now = 1;
    for(int i = 1; i <= t; ++i) {
        now = now * mi % modulo;
        if(table.contains(now)) {
            return i * t - table[now];
        }
    }
    return -1;
}
```

GF64

```
struct GF64 {
    GF64(ull _v = 0) : v(_v) {}
    GF64 operator+(GF64 o) const {
        return GF64(v ^ o.v);
    }
    GF64 &operator+=(GF64 o) {
        v ^= o.v;
        return *this;
    }
    GF64 operator*(GF64 o) const {
        ull a = v, b = o.v, r = 0;
        for(; b > 0; b >>= 1, a = (a << 1) ^ (a >> 63 ? 27 : 0))
            if((b & 1) == 1) r ^= a;
        return r;
    }
    GF64 qpow(ull e) const {
        GF64 r = 1, a = *this;
        for(; e > 0; e >>= 1, a = a * a)
            if((e & 1) == 1) r = r * a;
        return r;
    }
    GF64 inv() const {
        return qpow(~0ULL - 1);
    }
    ull val() const {
        return v;
    }

private:
    ull v;
};
```

Lehmer 码

```cpp
// Lehmer 码：l[i] 代表 #{ j > i | a[j] < a[i] }，其中 a 需要是一个排列
// 均为0-based排列
vector<int> lehmer(const vector<int> &a) {
    int n = a.size();
    vector<int> l(n);
    Fenwick<ll> bit(n);
    for(int i = 1; i <= n; ++i) {
        bit.update(i, 1);
    }
    for(int i = 0; i < n; ++i) {
        int x = a[i] + 1;
        l[i] = bit.query(n) - bit.query(x);
        bit.update(x, -1);
    }
    return l;
}
vector<int> rev_lehmer(const vector<int> &l) {
    int n = l.size();
    ValSeg vst(n); // 使用权值线段树实现会快一点
    for(int i = 0; i < n; ++i) {
        vst.insert(i);
    }
    vector<int> ret(n);
    for(int i = 0; i < n; ++i) {
        ret[i] = vst.qvr(l[i] + 1);
        vst.erase(ret[i]);
    }
    return ret;
}
```

类欧几里得算法

```
ll floor_sum(ll n, ll m, ll a, ll b) {
    ll ret = 0;
    while(true) {
        if(a >= m) {
            auto [d, r] = div(a, m);
            ret += n * (n - 1) / 2 * d;
            a = r;
        }
        if(b >= m) {
            auto [d, r] = div(b, m);
            ret += n * d;
            b = r;
        }
        ll ym = a * n + b;
        if(ym < m) break;
        n = ym / m;
        b = ym % m;
        swap(m, a);
    }
    return ret;
}
```

计算几何

基本数据类型

```
template<class T>
using opt = optional<T>;
constexpr auto nul = nullopt;

#ifndef eps
constexpr ld eps = 1e-9l;
#endif
constexpr ld pi = numbers::pi_v<ld>, inf = 1e12l;
int sign(ld a) {
    return (a < -eps) ? -1 : (a > eps) ? 1 : 0;
}
int cmp(ld a, ld b) {
    return sign(a - b);
}
auto cmpso(ld a, ld b) {
    return cmp(a, b) <=> 0;
}

struct Point {
    ld x, y;
    Point() : x(0.0l), y(0.0l) {}
    Point(ld _x, ld _y) : x(_x), y(_y) {}
    Point(const complex<ld> &cd) : x(cd.real()), y(cd.imag()) {}
    operator complex<ld>() const {
        return complex<ld>(x, y);
    }
    ld len2() const {
        return x * x + y * y;
    }
    ld len() const {
        return hypot(x, y);
    }
    Point norm() const {
        if(sign(len()) == 0) return Point();
        return (*this) * (1.0l / len());
    }
    // [0, 2pi)
    ld arg() const {
        ld ret = atan2(y, x);
        int c = cmp(ret, 0);
```

```
    return c == 1 ? ret : c == 0 ? 0.0l : ret + 2 * pi;
}
Point rotate(ld a) const {
    return Point(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
}
Point rotate(Point dir, bool slen = false) const {
    Point ret(x * dir.x - y * dir.y, x * dir.y + y * dir.x);
    if(slen) ret = ret / dir.len();
    return ret;
}
Point rotate90x(int n = 1) const {
    n %= 4;
    if(n < 0) n += 4;
    switch(n) {
    case 0:
        return *this;
    case 1:
        return Point(-y, x);
    case 2:
        return Point(-x, -y);
    default:
        return Point(y, -x);
    }
}
ld &operator[](int i) {
    return i == 0 ? x : y;
}
ld operator[](int i) const {
    return i == 0 ? x : y;
}
Point operator+(const Point &p) const {
    return Point(x + p.x, y + p.y);
}
Point operator-(const Point &p) const {
    return Point(x - p.x, y - p.y);
}
Point operator*(ld z) const {
    return Point(z * x, z * y);
}
friend Point operator*(ld z, const Point &p) {
    return p * z;
}
Point operator/(ld z) const {
    return Point(x / z, y / z);
}
```

```
    bool operator==(const Point &p) const {
        return cmp(x, p.x) == 0 && cmp(y, p.y) == 0;
    }
    auto operator<=>(const Point &p) const {
        auto cx = cmpso(x, p.x);
        if(cx != 0) return cx;
        return cmpso(y, p.y);
    }
};
ld dot(const Point &x, const Point &y) {
    return x.x * y.x + x.y * y.y;
}
ld cross(const Point &x, const Point &y) {
    return x.x * y.y - x.y * y.x;
}
ld cross(const Point &o, const Point &a, const Point &b) {
    return cross(a - o, b - o);
}
bool argcmp(const Point &x, const Point &y) {
    if(sign(x.x) == 0 && sign(x.y) == 0) return false;
    if(sign(y.x) == 0 && sign(y.y) == 0) return true;
    bool bx = sign(x.y) == 1 || (sign(x.y) == 0 && sign(x.x) == 1),
         by = sign(y.y) == 1 || (sign(y.y) == 0 && sign(y.x) == 1);
    if(bx != by) return bx;
    return sign(cross(x, y)) == 1;
}
ld dist(const Point &x, const Point &y) {
    return (x - y).len();
}
ld dist2(const Point &x, const Point &y) {
    return (x - y).len2();
}
int to_left(const Point &a, const Point &b) {
    return sign(cross(a, b));
}
int to_left(const Point &o, const Point &a, const Point &b) {
    return to_left(a - o, b - o);
}
ld angle(const Point &a, const Point &b) {
    ld cosa = clamp(dot(a, b) / (a.len() * b.len()), -1.0l, 1.0l);
    ld ret = acos(cosa);
    if(to_left(a, b) == -1) {
        ret = 2 * pi - ret;
    }
    return ret;
```

```
}

struct Line {
    Point p, v;
    Line() {}
    Line(const Point &_p, const Point &_v) : p(_p), v(_v) {}
    ld at(ld x) const {
        if(sign(v.x) == 0) return -inf;
        ld dx = x - p.x;
        ld rt = dx / v.x;
        return (p + rt * v).y;
    }
};
int to_left(const Line &ln, const Point &p) {
    if(p == ln.p) return 0;
    return to_left(ln.v, p - ln.p);
}
bool parallel(const Line &l1, const Line &l2) {
    return sign(cross(l1.v, l2.v)) == 0;
}
int is_inter(const Line &l1, const Line &l2) {
    return parallel(l1, l2) ? 0 : 1;
}
opt<Point> inter(const Line &a, const Line &b) {
    if(parallel(a, b)) return nul;
    Point v = a.v * (cross(b.v, a.p - b.p) / cross(a.v, b.v));
    return a.p + v;
}
ld dist(const Point &p, const Line &ln) {
    return abs(cross(ln.v, p - ln.p)) / ln.v.len();
}
ld dist(const Line &l1, const Line &l2) {
    if(!parallel(l1, l2)) return -1.0l;
    return dist(l1.p, l2);
}
Point proj(const Point &p, const Line &ln) {
    auto lv = ln.v.norm(), lp = ln.p;
    Point v = lv * (dot(lv, p - lp) / lv.len2());
    return lp + v;
}
bool is_on(const Point &p, const Line &ln) {
    return sign(cross(ln.v, ln.p - p)) == 0;
}
Line midperp(const Point &a, const Point &b) {
    Point mid = (a + b) / 2;
```

```
    Point to = (b - a).rotate90x();
    return Line(mid, to);
}

struct LineSeg {
    Point a, b;
    LineSeg() {}
    LineSeg(const Point &_a, const Point &_b) : a(_a), b(_b) {}
    ld len() const {
        return (b - a).len();
    }
    ld at(ld x) const {
        if(cmp(x, min(a.x, b.x)) == -1 || cmp(x, max(a.x, b.x)) == 1)
            return -inf;
        if(cmp(a.x, b.x) == 0) {
            if(cmp(a.x, x) != 0) return -inf;
            return max(a.y, b.y);
        }
        Line l(a, b - a);
        return l.at(x);
    }
};
int is_on(const Point &p, const LineSeg &ls) {
    if(p == ls.a || p == ls.b) return 2;
    return to_left(p - ls.a, p - ls.b) == 0 &&
           sign(dot(p - ls.a, p - ls.b)) == -1;
}
int is_inter(const Line &ln, const LineSeg &ls) {
    int a = to_left(ln, ls.a), b = to_left(ln, ls.b);
    if(a == 0 || b == 0) return 2;
    return a == b ? 0 : 1;
}
opt<Point> inter(const Line &ln, const LineSeg &ls) {
    if(!is_inter(ln, ls)) return nul;
    return inter(ln, Line(ls.a, ls.b - ls.a));
}
int is_inter(const LineSeg &l1, const LineSeg &l2) {
    if(is_on(l1.a, l2) || is_on(l1.b, l2) || is_on(l2.a, l1) || is_on(l2.b, l1))
        return 2;
    Line ln1(l1.a, l1.b - l1.a), ln2(l2.a, l2.b - l2.a);
    return to_left(ln1, l2.a) * to_left(ln1, l2.b) == -1 &&
           to_left(ln2, l1.a) * to_left(ln2, l1.b) == -1;
}
ld dist(const Point &p, const LineSeg &ls) {
    if(is_on(p, ls) != 0) return 0.0l;
```

```
    if(sign(dot(p - ls.a, ls.b - ls.a)) == -1 ||
       sign(dot(p - ls.b, ls.a - ls.b)) == -1)
        return min(dist(p, ls.a), dist(p, ls.b));
    Line l(ls.a, ls.b - ls.a);
    return dist(p, l);
}

vector<Point> poly_from_lines(const vector<Line> &ls) {
    vector<Point> poly;
    poly.reserve(ls.size());
    for(int i = 0; i < (int)ls.size(); ++i)
        poly.emplace_back(*inter(ls[i], ls[(i + 1) % ls.size()]));
    return poly;
}
ld poly_area(const vector<Point> &poly) {
    ld ret = 0.0l;
    for(int i = 0; i < (int)poly.size(); ++i)
        ret += cross(poly[i], poly[(i + 1) % poly.size()]);
    return ret / 2.0l;
}
ld poly_circ(const vector<Point> &poly) {
    ld ret = 0.0l;
    for(int i = 0; i < (int)poly.size(); ++i)
        ret += dist(poly[i], poly[(i + 1) % poly.size()]);
    return ret;
}

struct Circle {
    Point c;
    ld r;
    Circle() : r(0.0l) {}
    Circle(const Point &_c, ld _r) : c(_c), r(_r) {}
    ld area() const {
        return pi * r * r;
    }
    ld circ() const {
        return 2.0l * pi * r;
    }
};
bool is_in(const Circle &a, const Circle &b) {
    return sign(b.r - a.r - dist(a.c, b.c)) != -1;
}
bool is_in(const Point &p, const Circle &c) {
    return cmp(dist(p, c.c), c.r) <= 0;
}
```

```
bool is_inter(const Circle &c1, const Circle &c2) {
    ld d = (c2.c - c1.c).len();
    ld r1 = c1.r, r2 = c2.r;
    if(abs(d) <= eps) return false;
    if(d > r1 + r2 + eps || d < abs(r1 - r2) - eps) return false;
    return true;
}
opt<pair<Point, Point>> inter(const Circle &c1, const Circle &c2) {
    if(!is_inter(c1, c2)) return nul;
    ld d = dist(c2.c, c1.c);
    ld r1 = c1.r, r2 = c2.r;
    ld ca = clamp((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d), -1.0l, 1.0l);
    Point v = c2.c - c1.c;
    v = v / v.len() * c1.r;
    ld sa = sqrt(1 - ca * ca);
    Point rot1(ca, sa), rot2(ca, -sa);
    return pair{c1.c + v.rotate(rot1), c1.c + v.rotate(rot2)};
}
bool is_inter(const Circle &c, const Line &l) {
    ld d = dist(c.c, l);
    return cmp(c.r, d) != -1;
}
opt<pair<Point, Point>> inter(const Circle &c, const Line &l) {
    if(!is_inter(c, l)) return nul;
    Point h = proj(c.c, l);
    ld d2 = dist2(c.c, h);
    ld rem = max(c.r * c.r - d2, 0.0l);
    Point dir = l.v / l.v.len();
    ld t = sqrt(rem);
    Point p1 = h + dir * t;
    Point p2 = h - dir * t;
    return pair{p1, p2};
}
opt<pair<Point, Point>> tangent(const Circle &c, const Point &p) {
    Point v = p - c.c;
    ld d2 = v.len2();
    switch(cmp(d2, c.r * c.r)) {
    case -1:
        return nul;
    case 0:
        return pair{p, p};
    default: {
        Point base = c.c + v * c.r * c.r / d2;
        Point delt = v.rotate90x() * c.r * sqrt(d2 - c.r * c.r) / d2;
        return pair{base + delt, base - delt};
```

```
    }
    }
}
```

平面凸包

```
vector<Point> andrew(vector<Point> ps) {
    sort(ps.begin(), ps.end(), [](const Point &a, const Point &b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    });
    ps.erase(unique(ps.begin(), ps.end()), ps.end());
    int n = ps.size();
    if(n <= 2) return ps;
    vector<Point> stk;
    for(int i = 0; i < n; ++i) {
        while(stk.size() >= 2 &&
              sign(cross(stk[stk.size() - 2], stk.back(), ps[i])) <= 0)
            stk.pop_back();
        stk.push_back(ps[i]);
    }
    int t = stk.size();
    for(int i = n - 2; i >= 0; --i) {
        while((int)stk.size() > t &&
              sign(cross(stk[stk.size() - 2], stk.back(), ps[i])) <= 0)
            stk.pop_back();
        stk.push_back(ps[i]);
    }
    stk.pop_back();
    return stk;
}
```

最近点对

```
ld nearest(vector<Point> pts) {
    if(pts.size() == 1) return 0.0l;
    ld ans = 1e18l;
    sort(pts.begin(), pts.end(), [](const Point &a, const Point &b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    });
    auto part = [&](auto &&part, int l, int r) -> void {
        if(r - l < 5) {
            for(int i = l; i < r; ++i)
                for(int j = i + 1; j < r; ++j)
                    ans = min(ans, dist(pts[i], pts[j]));
            sort(pts.begin() + l, pts.begin() + r,
                 [](const Point &a, const Point &b) { return a.y < b.y; });
            return;
        }
        int m = (l + r) / 2;
        ld mx = pts[m].x;
        part(part, l, m);
        part(part, m, r);
        inplace_merge(pts.begin() + l, pts.begin() + m, pts.begin() + r,
                      [](const Point &a, const Point &b) { return a.y < b.y; });
        vector<Point> tmp;
        for(int i = l; i < r; ++i)
            if(abs(pts[i].x - mx) < ans) {
                for(int j = int(tmp.size()) - 1;
                    j >= 0 && pts[i].y - tmp[j].y < ans; --j)
                    ans = min(ans, dist(pts[i], tmp[j]));
                tmp.push_back(pts[i]);
            }
    };
    part(part, 0, pts.size());
    return ans;
}
```

半平面交

```
vector<Line> half_inter(vector<Line> ls) {
    ls.push_back({{-inf, 0.0l}, {0.0l, -1.0l}});
    ls.push_back({{inf, 0.0l}, {0.0l, 1.0l}});
    ls.push_back({{0.0l, inf}, {-1.0l, 0.0l}});
    ls.push_back({{0.0l, -inf}, {1.0l, 0.0l}});
    sort(ls.begin(), ls.end(), [](const Line &a, const Line &b) {
        int c = cmp(a.v.arg(), b.v.arg());
        if(c != 0) return c == -1;
        return sign(cross(a.v, b.p - a.p)) < 0;
    });
    deque<Line> dq;
    auto bad = [](const Line &l, const Line &b, const Line &c) {
        if(parallel(b, c)) return false;
        auto p = *inter(b, c);
        return to_left(l, p) < 0;
    };
    for(auto &l : ls) {
        if(!dq.empty() && cmp(l.v.arg(), dq.back().v.arg()) == 0) {
            if(to_left(l, dq.back().p + dq.back().v) < 0) dq.back() = l;
            continue;
        }
        while(dq.size() > 1 && bad(l, dq.back(), dq[dq.size() - 2]))
            dq.pop_back();
        while(dq.size() > 1 && bad(l, dq[0], dq[1])) dq.pop_front();
        dq.push_back(l);
    }
    while(dq.size() > 1 && bad(dq[0], dq.back(), dq[dq.size() - 2]))
        dq.pop_back();
    while(dq.size() > 1 && bad(dq.back(), dq[0], dq[1])) dq.pop_front();
    vector<Line> ret(dq.begin(), dq.end());
    int m = ret.size();
    return m < 3 ? vector<Line>() : ret;
}
```

扫描线

```
struct Rectangle {
    Point p1, p2;
};
struct Event {
    int d, l, r;
    ld y;
};
struct SegTree_SL {
    SegTree_SL(const vector<ld> &_xs)
        : l(4 * _xs.size()), r(4 * _xs.size()), cnt(4 * _xs.size()),
          len(4 * _xs.size()), xs(_xs) {
        int n = xs.size();
        [&](auto &&bld) {
            bld(bld, 0, 0, n);
        }([&](auto &&bld, int u, int l, int r) -> void {
            this->l[u] = l;
            this->r[u] = r;
            cnt[u] = 0;
            len[u] = 0.0l;
            if(r - l > 1) {
                int mid = (l + r) >> 1, lson = (u << 1) + 1,
                    rson = (u << 1) + 2;
                bld(bld, lson, l, mid);
                bld(bld, rson, mid, r);
            }
        });
    }
    ld query() const {
        return len[0];
    }
    // 注意左闭右开
    void update(int l, int r, int v) {
        [&](auto &&upd) {
            upd(upd, 0, l, r, v);
        }([&](auto &&upd, int u, int l, int r, int v) -> void {
            if(this->l[u] >= r || this->r[u] <= l) return;
            if(this->l[u] >= l && this->r[u] <= r) {
                cnt[u] += v;
                _pushup(u);
                return;
            }
            int lson = (u << 1) + 1, rson = (u << 1) + 2;
            upd(upd, lson, l, r, v);
```

```
            upd(upd, rson, l, r, v);
            _pushup(u);
        });
    }

private:
    vector<int> l, r, cnt;
    vector<ld> len, xs;
    void _pushup(int u) {
        if(cnt[u] > 0) {
            len[u] = xs[r[u]] - xs[l[u]];
        } else if(r[u] - l[u] == 1) {
            len[u] = 0.0l;
        } else {
            int lson = (u << 1) + 1, rson = (u << 1) + 2;
            len[u] = len[lson] + len[rson];
        }
    }
};
ld scanline(const vector<Rectangle> &rs) {
    int n = rs.size();
    vector<ld> xs(2 * n);
    vector<Event> evs(2 * n);
    for(int i = 0; i < n; ++i) {
        xs[2 * i] = rs[i].p1.x;
        xs[2 * i + 1] = rs[i].p2.x;
    }
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());
    auto getid = [&](ld x) -> int {
        return lower_bound(xs.begin(), xs.end(), x) - xs.begin();
    };
    for(int i = 0; i < n; ++i) {
        evs[2 * i] = {1, getid(rs[i].p1.x), getid(rs[i].p2.x), rs[i].p1.y};
        evs[2 * i + 1] = {-1, getid(rs[i].p1.x), getid(rs[i].p2.x), rs[i].p2.y};
    }
    sort(evs.begin(), evs.end(),
         [](const Event &a, const Event &b) { return cmp(a.y, b.y) == -1; });
    SegTree_SL seg(xs);
    ld ly = evs[0].y;
    ld ans = 0.0l;
    for(auto &e : evs) {
        ans += seg.query() * (e.y - ly);
        seg.update(e.l, e.r, e.d);
        ly = e.y;
```

```
    }
    return ans;
}
```

Pick定理

```
struct PointInt {
    ll x, y;
    PointInt() : x(0), y(0) {}
    PointInt(ll _x, ll _y) : x(_x), y(_y) {}
    PointInt operator+(const PointInt &p) const {
        return PointInt(x + p.x, y + p.y);
    }
    PointInt operator-(const PointInt &p) const {
        return PointInt(x - p.x, y - p.y);
    }
    PointInt operator*(ll z) const {
        return PointInt(z * x, z * y);
    }
    friend PointInt operator*(ll z, const PointInt &p) {
        return p * z;
    }
    bool operator==(const PointInt &p) const {
        return x == p.x && y == p.y;
    }
    ll len2() const {
        return x * x + y * y;
    }
    ll &operator[](int i) {
        return i == 0 ? x : y;
    }
    ll operator[](int i) const {
        return i == 0 ? x : y;
    }
};
ll dot(const PointInt &x, const PointInt &y) {
    return x.x * y.x + x.y * y.y;
}
ll cross(const PointInt &x, const PointInt &y) {
    return x.x * y.y - x.y * y.x;
}
ll cross(const PointInt &o, const PointInt &a, const PointInt &b) {
    return cross(a - o, b - o);
}
bool argcmp(const PointInt &x, const PointInt &y) {
    bool bx = x.y > 0 || (x.y == 0 && x.x > 0),
         by = y.y > 0 || (y.y == 0 && y.x > 0);
    if(bx != by) return bx;
    return cross(x, y) > 0;
```

```
}
ll dist2(const PointInt &a, const PointInt &b) {
    return (a - b).len2();
}
int to_left(const PointInt &a, const PointInt &b) {
    ll c = cross(a, b);
    return c > 0 ? 1 : c < 0 ? -1 : 0;
}
int to_left(const PointInt &a, const PointInt &b, const PointInt &c) {
    ll cr = cross(a, b, c);
    return cr > 0 ? 1 : cr < 0 ? -1 : 0;
}

struct LineInt {
    PointInt p, v;
    LineInt() {}
    LineInt(const PointInt &_p, const PointInt &_v) : p(_p), v(_v) {}
};
int to_left(const LineInt &ln, const PointInt &p) {
    return to_left(ln.v, p - ln.p);
}
bool parallel(const LineInt &l1, const LineInt &l2) {
    return cross(l1.v, l2.v) == 0;
}
int is_inter(const LineInt &l1, const LineInt &l2) {
    return parallel(l1, l2) ? 0 : 1;
}
bool is_on(const PointInt &p, const LineInt &ln) {
    return cross(ln.v, ln.p - p) == 0;
}

struct LineSegInt {
    PointInt a, b;
    LineSegInt() {}
    LineSegInt(const PointInt &_a, const PointInt &_b) : a(_a), b(_b) {}
};
int is_on(const PointInt &p, const LineSegInt &ls) {
    if(p == ls.a || p == ls.b) return 2;
    return to_left(p - ls.a, p - ls.b) == 0 && dot(p - ls.a, p - ls.b) < 0;
}
int is_inter(const LineInt &ln, const LineSegInt &ls) {
    int a = to_left(ln, ls.a), b = to_left(ln, ls.b);
    if(a == 0 || b == 0) return 2;
    return a == b ? 0 : 1;
}
```

```
int is_inter(const LineSegInt &l1, const LineSegInt &l2) {
    if(is_on(l1.a, l2) || is_on(l1.b, l2) || is_on(l2.a, l1) || is_on(l2.b, l1))
        return 2;
    LineInt ln1(l1.a, l1.b - l1.a), ln2(l2.a, l2.b - l2.a);
    return to_left(ln1, l2.a) * to_left(ln1, l2.b) == -1 &&
           to_left(ln2, l1.a) * to_left(ln2, l1.b) == -1;
}


ll polygon_twice_area(const vector<PointInt> &polygon) {
    ll ret = 0;
    for(int i = 0; i < (int)polygon.size(); ++i)
        ret += cross(polygon[i], polygon[(i + 1) % polygon.size()]);
    return abs(ret);
}
ll points_inside(const vector<PointInt> &polygon) {
    ll twos = polygon_twice_area(polygon);
    ll border = 0;
    for(int i = 0; i < (int)polygon.size(); ++i) {
        int j = (i + 1) % polygon.size();
        PointInt v = polygon[j] - polygon[i];
        border += gcd(abs(v.x), abs(v.y));
    }
    return (twos - border + 2) / 2;
}
```

圆与多边形相交

```
ld inter_area(const vector<Point> &poly, const Circle &c) {
    auto sec = [](Circle c, Point u, Point v) {
        u = u - c.c, v = v - c.c, c.c = {};
        ld alp = angle(u, v);
        if(cmp(alp, pi) == 1) alp -= 2 * pi;
        return c.r * c.r * alp / 2;
    };
    auto hlp = [&](Circle c, Point a, Point b) {
        a = a - c.c;
        b = b - c.c;
        c.c = {};
        ld da = a.len(), db = b.len();
        if(cmp(da, c.r) <= 0 && cmp(db, c.r) <= 0) return cross(a, b) / 2;
        ld di = dist(c.c, LineSeg{a, b});
        if(cmp(di, c.r) >= 0) return sec(c, a, b);
        auto [ia, ib] = *inter(c, Line{a, b - a});
        Point d = (b - a) / (b - a).len();
        ld ta = dot(ia - a, d), tb = dot(ib - a, d);
        if(ta > tb) swap(ia, ib);
        if(cmp(c.r, da) >= 0) return cross(a, ib) / 2 + sec(c, ib, b);
        else if(cmp(c.r, db) >= 0) return cross(ia, b) / 2 + sec(c, a, ia);
        else return cross(ia, ib) / 2 + sec(c, a, ia) + sec(c, ib, b);
    };
    ld ret = 0.0l;
    for(int i = 0; i < (int)poly.size(); ++i) {
        int j = (i + 1) % poly.size();
        ret += hlp(c, poly[i], poly[j]);
    }
    return abs(ret);
}
```

圆与圆相交

```
struct Arc {
    int idx;
    ld alpha, beta;
};
vector<Arc> circle_inter(const vector<Circle> &circs) {
    int n = circs.size();
    vector<Arc> ret;
    for(int i = 0; i < n; ++i) {
        const Circle &ci = circs[i];
        vector<pair<ld, ld>> forbidden;
        forbidden.reserve(2 * n);
        bool empty = false;
        for(int j = 0; j < n; ++j) {
            if(j == i) continue;
            const Circle &cj = circs[j];
            if(cmp(ci.r, 0.0l) == 0) {
                if(!is_in(ci.c, cj)) empty = true;
                continue;
            }
            if(is_in(ci, cj)) continue;
            if(!is_inter(ci, cj)) {
                empty = true;
                break;
            }

            ld d = dist(ci.c, cj.c);
            ld cosa =
                clamp((ci.r * ci.r + d * d - cj.r * cj.r) / (2 * ci.r * d),
                      -1.0l, 1.0l);
            ld a = acos(cosa);
            if(cmp(a, 0.0l) == 0) {
                empty = true;
                break;
            }
            ld start = (cj.c - ci.c).arg() + a;
            if(start >= 2 * pi) start -= 2 * pi;
            ld len = 2 * pi - 2 * a;
            if(cmp(len, 0.0l) == 0) continue;
            ld end = start + len;
            if(end <= 2 * pi) {
                forbidden.emplace_back(start, end);
            } else {
                forbidden.emplace_back(start, 2 * pi);
```

```
                forbidden.emplace_back(0.0l, end - 2 * pi);
            }
        }
        if(empty) continue;
        if(forbidden.empty()) {
            ret.push_back({i, 0.0l, 2 * pi});
            continue;
        }
        sort(forbidden.begin(), forbidden.end());
        vector<pair<ld, ld>> merged;
        for(auto [l, r] : forbidden) {
            if(merged.empty() || cmp(l, merged.back().second) == 1) {
                merged.emplace_back(l, r);
            } else {
                merged.back().second = max(merged.back().second, r);
            }
        }
        vector<Arc> arcs;
        for(int k = 1; k < (int)merged.size(); ++k) {
            ld l = merged[k - 1].second, r = merged[k].first;
            if(cmp(l, r) == -1) arcs.push_back({i, l, r});
        }
        ld l = merged.back().second, r = merged.front().first + 2 * pi;
        if(cmp(l, r) == -1) {
            if(cmp(l, 2 * pi) == 0)
                arcs.push_back({i, 0.0l, merged.front().first});
            else arcs.push_back({i, l, r});
        }
        sort(arcs.begin(), arcs.end(),
             [](const Arc &x, const Arc &y) { return x.alpha < y.alpha; });
        ret.insert(ret.end(), arcs.begin(), arcs.end());
    }
    return ret;
}
```

凸多边形

```
array<int, 3> convex_max_triangle(const vector<Point> &cv) {
    int m = cv.size();
    if(m < 3) return {-1, -1, -1};
    auto get = [&](int i) { return cv[i % m]; };
    ld max_area = -1.0l;
    array<int, 3> ret = {0, 1, 2};
    for(int i = 0; i < m; ++i) {
        int k = i + 2;
        for(int j = i + 1; j < i + m - 1; ++j) {
            while(k + 1 < i + m && cross(get(j) - get(i), get(k + 1) - get(i)) >
                                   cross(get(j) - get(i), get(k) - get(i)))
                ++k;
            ld area = 0.5l * cross(get(j) - get(i), get(k) - get(i));
            if(cmp(area, max_area) == 1) {
                max_area = area;
                ret = {i, j % m, k % m};
            }
        }
    }
    return ret;
}

pair<int, int> convex_farthest(const vector<Point> &cv) {
    int m = cv.size();
    if(m == 0) return {-1, -1};
    if(m == 1) return {0, 0};
    if(m == 2) return {0, 1};
    int j = 1;
    ld max_dist = 0;
    pair<int, int> ret = {0, 0};
    for(int i = 0; i < m; ++i) {
        int u = (i + 1) % m;
        while(cross(cv[i], cv[u], cv[(j + 1) % m]) > cross(cv[i], cv[u], cv[j]))
            j = (j + 1) % m;
        auto chmax = [&](int k) {
            ld now = (cv[k] - cv[j]).len2();
            if(cmp(now, max_dist) == 1) {
                max_dist = now;
                ret = {k, j};
            }
        };
        chmax(i);
        chmax(u);
```

```
    }
    return ret;
}

bool is_in_convex(const Point &pt, const vector<Point> &cv) {
    if(cv.empty()) return false;
    if(cv.size() == 1) return pt == cv[0];
    if(cv.size() == 2) return is_on(pt, LineSeg(cv[0], cv[1]));
    auto check = [](const Point &a, const Point &b, const Point &c,
                    const Point &p) {
        ld c1 = cross(b - a, p - a), c2 = cross(c - b, p - b),
           c3 = cross(a - c, p - c);
        return (sign(c1) != -1 && sign(c2) != -1 && sign(c3) != -1) ||
               (sign(c1) != 1 && sign(c2) != 1 && sign(c3) != 1);
    };
    int n = cv.size();
    Point pivot = cv[0];
    if(sign(cross(cv[1] - pivot, pt - pivot)) == -1 ||
       sign(cross(cv[n - 1] - pivot, pt - pivot)) == 1)
        return false;
    int l = 1, r = n - 1;
    while(l + 1 < r) {
        int mid = (l + r) >> 1;
        if(sign(cross(cv[mid] - pivot, pt - pivot)) != -1) l = mid;
        else r = mid;
    }
    int nxt = (l == n - 1) ? 1 : l + 1;
    return check(pivot, cv[l], cv[nxt], pt);
}

opt<pair<Point, Point>> convex_tangent(const Point &pt,
                                       const vector<Point> &cv) {
    if(is_in_convex(pt, cv)) return nul;
    int n = cv.size();
    auto peak = [&](int l, int r, bool f) {
        while(l < r - 1) {
            int m = (l + r) / 2;
            if(f ^ (to_left(cv[(m + n - 1) % n] - pt, cv[m] - pt) == 1)) r = m;
            else l = m;
        }
        return l;
    };
    if(to_left(cv[0] - pt, cv[1] - pt) == 0) {
        int idx = peak(2, n, cmp(dist(cv[0], pt), dist(cv[1], pt)) == -1);
        return pair{cv[0], cv[idx]};
```

```
    }
    bool al = true, ar = true;
    auto chk = [&](int x) {
        if(x == 1) ar = false;
        if(x == -1) al = false;
    };
    chk(to_left(cv[0] - pt, cv[1] - pt));
    chk(to_left(cv[0] - pt, cv[n - 1] - pt));
    if(al || ar) {
        int id = peak(1, n, al);
        return pair{cv[0], cv[id]};
    }
    int l = 1, r = n;
    while(l < r - 1) {
        int mid = (l + r) / 2;
        if(to_left(cv[0] - pt, cv[mid] - pt) == 1) l = mid;
        else r = mid;
    }
    bool f = to_left(cv[0] - pt, cv[1] - pt) == 1;
    int i1 = peak(0, l + 1, f), i2 = peak(l, n, !f);
    return pair{cv[i1], cv[i2]};
}

pair<Point, Point> convex_tangent(const Line &ln, const vector<Point> &cv) {
    int n = cv.size();
    Point dir(-ln.v.y, ln.v.x);
    auto find = [&](bool f) {
        int l = 0, r = n - 1;
        while(r - l > 2) {
            int m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
            ld dot1 = dot(cv[m1], dir), dot2 = dot(cv[m2], dir);
            if(f ^ (cmp(dot1, dot2) == -1)) r = m2;
            else l = m1;
        }
        int idx = l;
        ld ret = dot(cv[l], dir);
        for(int i = l; i <= r; ++i) {
            ld now = dot(cv[i], dir);
            if(f ^ (cmp(now, ret) == -1)) {
                idx = i;
                ret = now;
            }
        }
        return idx;
    };
```

```
    int i1 = find(true), i2 = find(false);
    return {cv[i1], cv[i2]};
}

vector<Point> minkowski_add(const vector<Point> &a, const vector<Point> &b) {
    int n = a.size(), m = b.size();
    auto cmp = [](const LineSeg &u, const LineSeg &v) {
        return argcmp(u.b - u.a, v.b - v.a);
    };
    vector<LineSeg> e1(n), e2(m), edge(n + m);
    for(int i = 0; i < n; ++i) e1[i] = {a[i], a[(i + 1) % n]};
    for(int i = 0; i < m; ++i) e2[i] = {b[i], b[(i + 1) % m]};
    rotate(e1.begin(), min_element(e1.begin(), e1.end(), cmp), e1.end());
    rotate(e2.begin(), min_element(e2.begin(), e2.end(), cmp), e2.end());
    merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
    vector<Point> ret;
    auto bad = [&](const Point &u) {
        Point b1 = ret.back(), b2 = *prev(ret.end(), 2);
        return to_left(b1 - b2, u - b1) == 0 && sign(dot(b1 - b2, u - b1)) >= 0;
    };
    auto u = e1[0].a + e2[0].a;
    for(const auto &v : edge) {
        while(ret.size() > 1 && bad(u)) ret.pop_back();
        ret.push_back(u);
        u = u + v.b - v.a;
    }
    if(ret.size() > 1 && bad(ret[0])) ret.pop_back();
    return ret;
}

vector<Point> minkowski_sub(const vector<Point> &a, const vector<Point> &b) {
    vector<Point> neg_b;
    neg_b.reserve(b.size());
    for(const auto &pt : b) neg_b.push_back(Point() - pt);
    return minkowski_add(a, neg_b);
}
```

KDTree

```
struct TwoDTree {
    explicit TwoDTree(const vector<Point> &pts) : val(pts.size()) {
        int n = val.size();
        for(int i = 0; i < n; ++i) {
            val[i].pt = pts[i];
            val[i].ls = val[i].rs = -1;
        }
        auto pushup = [&](int p) {
            for(int i = 0; i < 2; ++i) {
                val[p].L[i] = val[p].U[i] = val[p].pt[i];
                if(val[p].ls != -1) {
                    chkmin(val[p].L[i], val[val[p].ls].L[i]);
                    chkmax(val[p].U[i], val[val[p].ls].U[i]);
                }
                if(val[p].rs != -1) {
                    chkmin(val[p].L[i], val[val[p].rs].L[i]);
                    chkmax(val[p].U[i], val[val[p].rs].U[i]);
                }
            }
        };
        auto build = [&](auto &&build, int l, int r, int k) -> int {
            if(l >= r) return -1;
            int mid = (l + r) / 2;
            nth_element(val.begin() + l, val.begin() + mid, val.begin() + r,
                        [&](const Node &a, const Node &b) {
                            return a.pt[k] < b.pt[k];
                        });
            val[mid].ls = build(build, l, mid, 1 - k);
            val[mid].rs = build(build, mid + 1, r, 1 - k);
            pushup(mid);
            return mid;
        };
        rt = build(build, 0, n, 0);
    }
    template<class Comp>
    ld kth_cmp(int k) {
        k *= 2;
        priority_queue<ld, vector<ld>, Comp> pq;
        for(int i = 0; i < val.size(); ++i) {
            kth(rt, val[i].pt, pq, k, 0);
        }
        return sqrt(pq.top());
    }
```

```
    ld kth_nearest(int k) {
        return kth_cmp<less<>>(k);
    }
    ld kth_farthest(int k) {
        return kth_cmp<greater<>>(k);
    }
    template<class Comp>
    ld kth_cmp(const Point &p, int k) {
        k *= 2;
        priority_queue<ld, vector<ld>, Comp> pq;
        kth(rt, p, pq, k, 0);
        return sqrt(pq.top());
    }
    ld kth_nearest(const Point &p, int k) {
        return kth_cmp<less<>>(p, k);
    }
    ld kth_farthest(const Point &p, int k) {
        return kth_cmp<greater<>>(p, k);
    }

private:
    struct Node {
        int ls, rs;
        Point pt;
        ld L[2], U[2];
    };
    vector<Node> val;
    int rt;
    template<class Comp>
    ld cmpdist(const Point &p, const Node &node) = delete;
    template<class Comp>
    void kth(int p, const Point &q, priority_queue<ld, vector<ld>, Comp> &pq,
             int k, int dep) {
        static Comp comp;
        if(p == -1) return;
        ld dist = (val[p].pt - q).len2();
        if(dist > 1e-12) {
            if(pq.size() < k) {
                pq.push(dist);
            } else if(comp(dist, pq.top())) {
                pq.pop();
                pq.push(dist);
            }
        }
        int dim = dep % 2;
```

```
        int fi, se;
        if(q[dim] < val[p].pt[dim]) {
            fi = val[p].rs;
            se = val[p].ls;
        } else {
            fi = val[p].ls;
            se = val[p].rs;
        }
        if(fi != -1) {
            kth(fi, q, pq, k, dep + 1);
        }
        if(se != -1) {
            ld md = cmpdist<Comp>(q, val[se]);
            if(comp(md, pq.size() < k ? inf : pq.top()) || pq.size() < k) {
                kth(se, q, pq, k, dep + 1);
            }
        }
    }
};

template<>
ld TwoDTree::cmpdist<less<>>(const Point &p, const Node &node) {
    ld d = 0;
    for(int i = 0; i < 2; ++i) {
        if(p[i] < node.L[i]) {
            ld t = node.L[i] - p[i];
            d += t * t;
        } else if(p[i] > node.U[i]) {
            ld t = p[i] - node.U[i];
            d += t * t;
        }
    }
    return d;
}
template<>
ld TwoDTree::cmpdist<greater<>>(const Point &p, const Node &node) {
    ld dx = max(abs(p.x - node.L[0]), abs(p.x - node.U[0]));
    ld dy = max(abs(p.y - node.L[1]), abs(p.y - node.U[1]));
    return dx * dx + dy * dy;
}
```

李超线段树

```
struct LiChaoInt {
    vector<array<ll, 4>> raw;
    explicit LiChaoInt(int ma) : id(ma * 4, -1), n(ma) {}
    void addline(int x1, int x2, int y1, int y2) {
        if(x1 > x2) {
            swap(x1, x2);
            swap(y1, y2);
        }
        int i = raw.size();
        raw.push_back({x1, x2, y1, y2});
        [&](auto &&upd) {
            upd(upd, i, min(x1, x2), max(x1, x2), 0, 0, n - 1);
        }([&](auto &&upd, int i, int ul, int ur, int u, int rl,
              int rr) -> void {
            int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
            if(ul <= rl && ur >= rr) {
                if(id[u] == -1) {
                    id[u] = i;
                    return;
                }
                if(_better(i, id[u], mid)) swap(i, id[u]);
                if(rl == rr) return;
                if(_better(i, id[u], rl)) upd(upd, i, ul, ur, ls, rl, mid);
                if(_better(i, id[u], rr)) upd(upd, i, ul, ur, rs, mid + 1, rr);
            } else {
                if(rl == rr) return;
                if(ul <= mid) upd(upd, i, ul, ur, ls, rl, mid);
                if(mid < ur) upd(upd, i, ul, ur, rs, mid + 1, rr);
            }
        });
    }
    int query(int k) const {
        return [&](auto &&qry) {
            return qry(qry, k, 0, 0, n - 1);
        }([&](auto &&qry, int k, int u, int rl, int rr) -> int {
            int ret = id[u];
            if(rl == rr) return ret;
            int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
            int child = (k <= mid) ? qry(qry, k, ls, rl, mid)
                                   : qry(qry, k, rs, mid + 1, rr);
            if(_better(child, ret, k)) ret = child;
            return ret;
        });
```

```
    }

private:
    vector<int> id;
    int n;
    pair<__int128, __int128> _calc(int i, int x) const {
        auto [x1, x2, y1, y2] = raw[i];
        if(x1 == x2) return {max(y1, y2), 1};
        __int128 den = x2 - x1;
        __int128 num = (__int128)y1 * den + (__int128)(y2 - y1) * (x - x1);
        return {num, den};
    }
    bool _better(int a, int b, int x) const {
        if(b == -1) return true;
        if(a == -1) return false;
        auto [na, da] = _calc(a, x);
        auto [nb, db] = _calc(b, x);
        __int128 lhs = na * db, rhs = nb * da;
        if(lhs != rhs) return lhs > rhs;
        return a < b;
    }
};
```

字符串

KMP

```
// 得到用于KMP的数组
vector<int> kmp_f(const string &s) {
    int n = s.size();
    vector<int> f(n, 0);
    for(int i = 1, j = 0; i < n; ++i) {
        while(j > 0 && s[i] != s[j]) j = f[j - 1];
        if(s[i] == s[j]) ++j;
        f[i] = j;
    }
    return f;
}
// 寻找第一次在off后面haystack中的needle
int kmp_find(const string &haystack, const string &needle, int off = 0) {
    int n = needle.size(), m = haystack.size();
    vector<int> f = kmp_f(needle);
    for(int i = 0, j = off; j < m;) {
        if(haystack[j] == needle[i]) {
            ++i;
            ++j;
            if(i == n) return j - n;
        } else {
            (i == 0) ? (++j) : (i = f[i - 1]);
        }
    }
    return -1;
}
// 返回haystack中有多少个needle，可重（即10101有两个101）
int kmp_count(const string &haystack, const string &needle) {
    int n = needle.size(), m = haystack.size();
    int ret = 0;
    vector<int> f = kmp_f(needle);
    for(int i = 0, j = 0; j < m;) {
        if(haystack[j] == needle[i]) {
            ++i;
            ++j;
            if(i == n) {
                ++ret;
                i = f[i - 1];
            }
        } else {
```

```
            (i == 0) ? (++j) : (i = f[i - 1]);
        }
    }
    return ret;
}
```

Manacher

```
vector<int> manacher(const string &_s) {
    string s = "$";
    for(char ch : _s) {
        s += ch;
        s += '$';
    }
    int n = s.size();
    int mr = 0;
    int c = 0;
    vector<int> dp(n, 0);
    auto expand = [&](int l, int r) {
        while(l >= 0 && r < n && s[l] == s[r]) {
            --l;
            ++r;
        }
        return (r - l) / 2;
    };
    for(int i = 0; i < n; ++i) {
        int m = 2 * c - i;
        if(i >= mr) {
            dp[i] = expand(i, i);
            mr = i + dp[i];
            c = i;
        } else if(dp[m] == mr - i) {
            dp[i] = expand(i - dp[m], i + dp[m]);
            mr = i + dp[i];
            c = i;
        } else {
            dp[i] = min(dp[m], mr - i);
        }
    }
    return dp;
}
```

字典树

```
template<int N, class F>
struct Trie {
    Trie() : nxt(1), end(1), cnt(1) {
        nxt[0].fill(-1);
    }
    void insert(const string &str) {
        int rt = 0;
        for(int i = 0; i < str.size(); ++i) {
            int id = mapper(str[i]);
            if(nxt[rt][id] == -1) {
                nxt[rt][id] = size();
                nxt.emplace_back();
                nxt.back().fill(-1);
                end.push_back(false);
                cnt.push_back(0);
            }
            cnt[rt]++;
            rt = nxt[rt][id];
        }
        cnt[rt]++;
        end[rt] = true;
    }
    bool find(const string &str) const {
        int rt = 0;
        for(int i = 0; i < str.size(); ++i) {
            int id = mapper(str[i]);
            if(nxt[rt][id] == -1) {
                return false;
            }
            rt = nxt[rt][id];
        }
        return end[rt];
    }
    int prefix_count(const string &str) const {
        int rt = 0;
        for(int i = 0; i < str.size(); ++i) {
            int id = mapper(str[i]);
            if(nxt[rt][id] == -1) {
                return 0;
            }
            rt = nxt[rt][id];
        }
        return cnt[rt];
```

```
    }

private:
    vector<array<int, N>> nxt;
    vector<bool> end;
    vector<int> cnt;
    F mapper;
    int size() const {
        return nxt.size();
    }
};
```

字符串哈希

```
struct StringHash {
    explicit StringHash(const string &s)
        : p1(s.size() + 1, 0), p2(s.size() + 1, 0) {
        for(int i = 0; i < (int)s.size(); ++i) {
            p1[i + 1] = (p1[i] * _s.mul1 + s[i]) % modulo;
            p2[i + 1] = p2[i] * _s.mul2 + s[i];
        }
    }
    ull substr(int l, int r) const {
        ull ret1 = (p1[r] - p1[l] * _s.pmul1[r - l] % modulo + modulo) % modulo;
        ull ret2 = p2[r] - p2[l] * _s.pmul2[r - l];
        return (ret1 << 3) ^ (ret1 >> 5) ^ ret2;
    }

private:
    vector<ull> p1, p2;
    static constexpr ull modulo = 998244353;
    static constexpr int maxn = 500005;
    struct _Statics {
        ull mul1, mul2;
        ull pmul1[maxn], pmul2[maxn];
        _Statics() {
            ull c = (ull)chrono::steady_clock::now().time_since_epoch().count();
            mul1 = c % 131 + 131;
            mul2 = c % 13331 + 13331;
            pmul1[0] = pmul2[0] = 1;
            for(int i = 1; i < maxn; ++i) {
                pmul1[i] = pmul1[i - 1] * mul1 % modulo;
                pmul2[i] = pmul2[i - 1] * mul2;
            }
        }
    };
    static inline _Statics _s;
};
```

AC自动机

```
struct ACAM {
    ACAM() : nxt(1), fail(1, 0) {
        nxt[0].fill(-1);
    }
    void insert(const string &str) {
        int rt = 0;
        for(int i = 0; i < (int)str.size(); ++i) {
            int id = str[i] - '0';
            if(nxt[rt][id] == -1) {
                nxt[rt][id] = size();
                nxt.emplace_back();
                nxt.back().fill(-1);
                fail.push_back(0);
            }
            rt = nxt[rt][id];
        }
    }
    void build() {
        queue<int> q;
        for(int i = 0; i < 2; ++i) {
            int v = nxt[0][i];
            if(v != -1) {
                fail[v] = 0;
                q.push(v);
            } else {
                nxt[0][i] = 0;
            }
        }
        while(!q.empty()) {
            int rt = q.front();
            q.pop();
            for(int i = 0; i < 2; ++i) {
                int v = nxt[rt][i];
                if(v != -1) {
                    fail[v] = nxt[fail[rt]][i];
                    q.push(v);
                } else {
                    nxt[rt][i] = nxt[fail[rt]][i];
                }
            }
        }
    }
    vector<array<int, 2>> nxt;
```

```
    vector<int> fail;
    int size() const {
        return nxt.size();
    }
};
```

后缀数组

```
vector<int> suffix_array(const string &str) {
    int n = str.size(), m = 128, p = 0;
    vector<int> rk(n * 3, -1), sa(n), id(n), cnt(max(n, m), 0);
    for(int i = 0; i < n; ++i) cnt[rk[i] = str[i]]++;
    for(int i = 1; i < m; ++i) cnt[i] += cnt[i - 1];
    for(int i = n - 1; i >= 0; --i) sa[--cnt[rk[i]]] = i;
    for(int w = 1;; w <<= 1, m = p + 1) {
        int cur = 0;
        for(int i = n - w; i < n; ++i) id[cur++] = i;
        for(int i = 0; i < n; ++i) {
            if(sa[i] >= w) id[cur++] = sa[i] - w;
        }
        cnt.assign(max(n, m), 0);
        for(int i = 0; i < n; ++i) cnt[rk[i]]++;
        for(int i = 1; i < m; ++i) cnt[i] += cnt[i - 1];
        for(int i = n - 1; i >= 0; --i) sa[--cnt[rk[id[i]]]] = id[i];
        p = 0;
        vector<int> oldrk = rk;
        rk[sa[0]] = 0;
        for(int i = 1; i < n; ++i) {
            if(oldrk[sa[i]] != oldrk[sa[i - 1]] ||
               oldrk[sa[i] + w] != oldrk[sa[i - 1] + w])
                ++p;
            rk[sa[i]] = p;
        }
        if(p == n - 1) break;
    }
    return sa;
}
```

Height数组

```
vector<int> height(const string &str) {
    int n = str.size();
    vector<int> sa = suffix_array(str), rk(n), h(n);
    for(int i = 0; i < n; ++i) rk[sa[i]] = i;
    for(int i = 0, k = 0; i < n; ++i) {
        if(rk[i] == 0) continue;
        if(k > 0) --k;
        while(str[i + k] == str[sa[rk[i] - 1] + k]) ++k;
        h[rk[i]] = k;
    }
    return h;
}
```

后缀自动机

```
struct SAM {
    SAM() : nxt(1), link(1, -1), len(1, 0), last(0) {
        nxt[0].fill(-1);
    }
    void insert(char ch) {
        const int id = ch - 'a';
        const int cur = size();
        nxt.emplace_back();
        nxt.back().fill(-1);
        link.push_back(-1);
        len.push_back(len[last] + 1);
        int p = last;
        while(p != -1 && nxt[p][id] == -1) {
            nxt[p][id] = cur;
            p = link[p];
        }
        if(p == -1) {
            link[cur] = 0;
        } else {
            int q = nxt[p][id];
            if(len[p] + 1 == len[q]) {
                link[cur] = q;
            } else {
                int clone = size();
                nxt.push_back(nxt[q]);
                link.push_back(link[q]);
                len.push_back(len[p] + 1);
                while(p != -1 && nxt[p][id] == q) {
                    nxt[p][id] = clone;
                    p = link[p];
                }
                link[q] = link[cur] = clone;
            }
        }
        last = cur;
        ends.push_back(cur);
    }
    ll solve() const {
        int n = size();
        vector<int> indeg(n, 0), topoorder;
        topoorder.reserve(n);
        for(int i = 0; i < n; ++i) {
            if(link[i] != -1) {
```

```
                indeg[link[i]]++;
            }
        }
        queue<int> q;
        for(int i = 0; i < n; ++i) {
            if(indeg[i] == 0) {
                q.push(i);
            }
        }
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            topoorder.push_back(u);
            if(link[u] != -1) {
                indeg[link[u]]--;
                if(indeg[link[u]] == 0) {
                    q.push(link[u]);
                }
            }
        }
        ll ans = 0;
        vector<ll> occur(n, 0);
        for(int e : ends) {
            occur[e] = 1;
        }
        for(int i : topoorder) {
            if(link[i] != -1) {
                occur[link[i]] += occur[i];
            }
            if(occur[i] != 1) {
                ans = max(ans, occur[i] * len[i]);
            }
        }
        return ans;
    }
    vector<array<int, 26>> nxt;
    vector<int> link, len;
    vector<int> ends;
    int last;
    int size() const {
        return nxt.size();
    }
};
```

Lyndon分解

```
vector<string> duval(const string &s) {
    int n = s.size();
    vector<string> ret;
    for(int i = 0; i < n;) {
        int j = i + 1, k = i;
        while(j < n && s[k] <= s[j]) {
            (s[k] < s[j]) ? (k = i) : (k++);
            ++j;
        }
        while(i <= k) {
            ret.push_back(s.substr(i, j - k));
            i += (j - k);
        }
    }
    return ret;
}
```

最小表示法

```
string minrep(const string &str) {
    int n = str.size(), i = 0, j = 1, k = 0;
    string s = str + str;
    while(i < n && j < n) {
        while(k < n && s[i + k] == s[j + k]) ++k;
        if(k == n) break;
        if(s[i + k] > s[j + k]) i += (k + 1);
        else j += (k + 1);
        if(i == j) ++j;
        k = 0;
    }
    return s.substr(min(i, j), n);
}
```

序列自动机

```
struct SeqAM {
    explicit SeqAM(const string &s)
        : n(s.size()), nxt(s.size() + 2, [&] {
              array<int, 26> ret;
              ret.fill(s.size() + 1);
              return ret;
          }()) {
        for(int i = n - 1; i >= 0; --i) {
            nxt[i] = nxt[i + 1];
            nxt[i][s[i] - 'a'] = i + 1;
        }
    }
    bool match(const string &t) const {
        int now = 0;
        for(char c : t) {
            now = nxt[now][c - 'a'];
        }
        return now != (n + 1);
    }
    int next(int s, char c) const {
        return nxt[s][c - 'a'];
    }

private:
    int n;
    vector<array<int, 26>> nxt;
};
```

回文自动机

```
struct PAM {
    PAM() : s("#"), last(1) {
        nxt.emplace_back();
        nxt.back().fill(-1);
        len.push_back(-1);
        fail.push_back(0);
        cnt.push_back(0);
        nxt.emplace_back();
        nxt.back().fill(-1);
        len.push_back(0);
        fail.push_back(0);
        cnt.push_back(0);
    }
    int insert(char ch) {
        s += ch;
        int pos = s.size() - 1;
        int p = last, id = ch - 'a';
        while(s[pos - len[p] - 1] != ch) p = fail[p];
        if(nxt[p][id] == -1) {
            int cur = size();
            nxt[p][id] = cur;
            nxt.emplace_back();
            nxt.back().fill(-1);
            len.push_back(len[p] + 2);
            fail.push_back(0);
            cnt.push_back(0);
            if(len[cur] == 1) {
                fail[cur] = 1;
            } else {
                int f = fail[p];
                while(s[pos - len[f] - 1] != ch) f = fail[f];
                fail[cur] = nxt[f][id];
            }
            cnt[cur] = len[cur] == 1 ? 1 : cnt[fail[cur]] + 1;
        }
        last = nxt[p][id];
        return cnt[last];
    }
    vector<int> len, fail, cnt;
    vector<array<int, 26>> nxt;
    string s;
    int last;
    int size() const {
```

```
        return len.size();
    }
};
```

Z函数

```
vector<int> zfn(const string &s) {
    int n = s.size();
    vector<int> z(n);
    for(int i = 1, l = 0, r = 0; i < n; ++i) {
        if(i <= r && z[i - l] < r - i + 1) {
            z[i] = z[i - l];
        } else {
            z[i] = max(0, r - i + 1);
            while(i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        }
        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
```

其他

最长上升子序列

```
vector<int> lis(const vector<int> &nums) {
    int n = nums.size();
    vector<int> dp, pos(n), pre(n, -1);
    for(int i = 0; i < n; ++i) {
        auto it = lower_bound(dp.begin(), dp.end(), nums[i]);
        int j = it - dp.begin();
        if(it == dp.end()) dp.push_back(nums[i]);
        else *it = nums[i];
        pos[j] = i;
        if(j != 0) pre[i] = pos[j - 1];
    }
    vector<int> ret;
    for(int i = pos[dp.size() - 1]; i != -1; i = pre[i]) {
        ret.push_back(nums[i]);
    }
    reverse(ret.begin(), ret.end());
    return ret;
}
```

高精度整数

```
struct HPINT {
    HPINT() : nums(1, 0), neg(false) {}
    // intentionally implicit, allowing HPINT a = 5
    HPINT(ll val) : neg(false) {
        if(val < 0) {
            neg = true;
            val = -val;
        }
        while(val) {
            int t = val % 10;
            nums.push_back(t);
            val /= 10;
        }
        if(nums.empty()) nums = vector<int>(1, 0);
    }
    // and HPINT b = "1145141919810314159265358979323846262171828 1828"
    HPINT(const string &s) : neg(false) {
        int off = 0, n = s.size();
        if(s[0] == '-') {
            off = 1;
            neg = true;
        }
        for(; off < n; ++off) nums.push_back(s[off] - '0');
        reverse(nums.begin(), nums.end());
        _norm();
    }
    explicit operator string() const {
        string ret;
        if(neg) ret += '-';
        for(int i = nums.size() - 1; i >= 0; --i) ret += (nums[i] + '0');
        return ret;
    }
    HPINT &operator+=(const HPINT &r) {
        if(neg) {
            if(r.neg) {
                _add(nums, r.nums);
            } else if(_cmp(nums, r.nums) == 1) {
                _sub(nums, r.nums);
            } else {
                vector<int> rnums = r.nums;
                _sub(rnums, nums);
                nums.swap(rnums);
                neg = false;
```

```
        }
    } else if(r.neg) {
        if(_cmp(nums, r.nums) != -1) {
            _sub(nums, r.nums);
        } else {
            vector<int> rnums = r.nums;
            _sub(rnums, nums);
            nums.swap(rnums);
            neg = true;
        }
    } else {
        _add(nums, r.nums);
    }
    _norm();
    return *this;
}
HPINT &operator-=(const HPINT &r) {
    if(neg) {
        if(r.neg) {
            if(_cmp(nums, r.nums) == 1) {
                _sub(nums, r.nums);
            } else {
                vector<int> rnums = r.nums;
                _sub(rnums, nums);
                nums.swap(rnums);
                neg = false;
            }
        } else {
            _add(nums, r.nums);
        }
    } else if(r.neg) {
        _add(nums, r.nums);
    } else {
        int c = _cmp(nums, r.nums);
        if(c == -1) {
            vector<int> rnums = r.nums;
            _sub(rnums, nums);
            nums.swap(rnums);
            neg = true;
        } else {
            _sub(nums, r.nums);
        }
    }
    _norm();
    return *this;
```

```
}
HPINT &operator*=(const HPINT &r) {
    neg ^= r.neg;
    _mul(nums, r.nums);
    _norm();
    return *this;
}
friend ostream &operator<<(ostream &os, const HPINT &hp) {
    if(hp.neg) os << '-';
    for(int i = hp.nums.size() - 1; i >= 0; --i) os << hp.nums[i];
    return os;
}
friend istream &operator>>(istream &is, HPINT &hp) {
    char ch = is.rdbuf()->sbumpc();
    while(ch < '0' || ch > '9') {
        if(ch == '-') hp.neg = true;
        ch = is.rdbuf()->sbumpc();
    }
    string buffer;
    while(ch >= '0' && ch <= '9') {
        buffer += ch;
        ch = is.rdbuf()->sbumpc();
    }
    hp.nums.resize(buffer.size());
    int n = buffer.size();
    for(int i = 0; i < n; ++i) hp.nums[i] = buffer[n - i - 1] - '0';
    hp._norm();
    return is;
}
bool operator==(const HPINT &r) const {
    return neg == r.neg && _cmp(nums, r.nums) == 0;
}
strong_ordering operator<=>(const HPINT &r) const {
    if(neg != r.neg) {
        if(neg) return strong_ordering::less;
        return strong_ordering::greater;
    }
    int c = _cmp(nums, r.nums);
    if(neg) c *= -1;
    return c <=> 0;
}
HPINT operator+(const HPINT &r) const {
    HPINT ret = *this;
    ret += r;
    return ret;
```

```
    }
    HPINT operator-(const HPINT &r) const {
        HPINT ret = *this;
        ret -= r;
        return ret;
    }
    HPINT operator*(const HPINT &r) const {
        HPINT ret = *this;
        ret *= r;
        return ret;
    }

private:
    vector<int> nums;
    bool neg;
    void _norm() {
        while(nums.size() > 1 && nums.back() == 0) nums.pop_back();
        if(nums.size() == 1 && nums[0] == 0) neg = false;
    }
    static void _add(vector<int> &a, const vector<int> &b) {
        int carry = 0;
        for(int i = 0; i < b.size(); ++i) {
            if(i >= a.size()) a.push_back(0);
            a[i] += (b[i] + carry);
            carry = 0;
            if(a[i] >= 10) {
                carry = 1;
                a[i] -= 10;
            }
        }
        int i = b.size();
        while(carry != 0) {
            if(i >= a.size()) a.push_back(0);
            a[i] += carry;
            carry = 0;
            if(a[i] >= 10) {
                carry = 1;
                a[i] -= 10;
            }
            ++i;
        }
    }
    // we are assuming that a >= b
    static void _sub(vector<int> &a, const vector<int> &b) {
        int borrow = 0;
```

```
    for(int i = 0; i < b.size(); ++i) {
        a[i] -= (borrow + b[i]);
        borrow = 0;
        if(a[i] < 0) {
            a[i] += 10;
            borrow = 1;
        }
    }
    int i = b.size();
    while(borrow && i < a.size()) {
        a[i] -= borrow;
        borrow = 0;
        if(a[i] < 0) {
            a[i] += 10;
            borrow = 1;
        }
        ++i;
    }
    while(a.size() > 1 && a.back() == 0) a.pop_back();
}
static void _mul(vector<int> &a, const vector<int> &b) {
    using cd = complex<double>;
    constexpr double pi = 3.14159265358979323846264338327950288;
    int n = 1;
    while(n < (int)(a.size() + b.size())) n <<= 1;
    vector<cd> fa(n), fb(n);
    for(int i = 0; i < (int)a.size(); ++i) fa[i] = a[i];
    for(int i = 0; i < (int)b.size(); ++i) fb[i] = b[i];
    auto fft = [](auto &&fft, vector<cd> &f, bool invert) -> void {
        int n = f.size();
        if(n == 1) return;
        vector<cd> f0(n / 2), f1(n / 2);
        for(int i = 0; i < n / 2; ++i) {
            f0[i] = f[2 * i];
            f1[i] = f[2 * i + 1];
        }
        fft(fft, f0, invert);
        fft(fft, f1, invert);
        double theta = 2 * pi / n * (invert ? -1 : 1);
        cd wt = 1, w(cos(theta), sin(theta));
        for(int t = 0; t < n / 2; ++t) {
            cd u = f0[t], v = wt * f1[t];
            f[t] = u + v;
            f[t + n / 2] = u - v;
            wt *= w;
```

```
            }
        };
        fft(fft, fa, false);
        fft(fft, fb, false);
        for(int i = 0; i < n; ++i) fa[i] *= fb[i];
        fft(fft, fa, true);
        a.resize(n);
        int carry = 0;
        for(int i = 0; i < n; ++i) {
            int num = int(fa[i].real() / n + 0.5) + carry;
            carry = num / 10;
            a[i] = num % 10;
        }
        while(carry) {
            a.push_back(carry % 10);
            carry /= 10;
        }
        while(a.size() > 1 && a.back() == 0) a.pop_back();
    }
    // 1 gt, -1 lt, 0 eq
    static int _cmp(const vector<int> &a, const vector<int> &b) {
        if(a.size() > b.size()) return 1;
        if(a.size() < b.size()) return -1;
        for(int n = a.size(), i = n - 1; i >= 0; --i) {
            if(a[i] > b[i]) return 1;
            if(a[i] < b[i]) return -1;
        }
        return 0;
    }
};
```

快速读入

```
struct Qread {
    Qread() : state(true) {
        cin.rdbuf()->pubsetbuf(buffer, maxn);
    }
    int getc() {
        return cin.rdbuf()->sbumpc();
    }
    template<integral T>
    Qread &operator>>(T &val) {
        if(!state) {
            val = 0;
            return *this;
        }
        T x = 0, f = 1;
        char ch = getc();
        while(ch < '0' || ch > '9') {
            if(ch == char_traits<char>::eof()) {
                state = false;
                cin.setstate(ios_base::eofbit);
                val = x * f;
                return *this;
            }
            if(ch == '-') {
                f = -1;
            }
            ch = getc();
        }
        while(ch >= '0' && ch <= '9') {
            x = x * 10 + ch - '0';
            ch = getc();
        }
        val = x * f;
        return *this;
    }
    Qread &operator>>(string &val) {
        val.clear();
        if(!state) {
            return *this;
        }
        int ch = getc();
        while(ch != char_traits<char>::eof() && isspace(ch)) {
            ch = getc();
        }
```

```
        if(ch == char_traits<char>::eof()) {
            state = false;
            cin.setstate(ios_base::eofbit);
            return *this;
        }
        while(ch != char_traits<char>::eof() && !isspace(ch)) {
            val.push_back(char(ch));
            ch = getc();
        }
        return *this;
    }
    explicit operator bool() const {
        return state;
    }

private:
    bool state;
    static constexpr int maxn = 1 << 21;
    char buffer[maxn];
} qread;
```

SplitMix64 自定义哈希

```
struct MyHash {
    ull operator()(ll x) const noexcept {
        ull v = ull(x) + c;
        v += 0x9e3779b97f4a7c15;
        v = (v ^ (v >> 30)) * 0xbf58476d1ce4e5b9;
        v = (v ^ (v >> 27)) * 0x94d049bb133111eb;
        return v ^ (v >> 31);
    }

private:
    static inline const ull c =
        chrono::steady_clock::now().time_since_epoch().count();
};
```